Programming Language Foundations
Appunti

Alex "Morbidelli WhatsApp" Basta

Contents

Chapter 1 **Operational Semantics** Page

1.1 Properties of programs
Inductive Sets — • Inductive Proofs —
1.2 Large-step Operational Semantics
1.3 Equivalence of semantics
1.4 A simple imperative language IMP
Small-step operational semantics —
1.5 Large-step operational semantics for IMP
Command equivalence —

Chapter 2 **Denotational Semantics** Page

2.1 A Denotational Semantics for IMP
2.2 Fixed Points
2.3 Denotation for Loops
2.4 Kleene's Fixed Point Theorem
2.5 Reasoning using Denotational Semantics

Chapter 3 **Axiomatic Semantics** Page

3.1 Introduction to axiomatic semantics
3.2 Assertions
3.3 Satisfaction and Validity
3.4 Hoare Logic
3.5 Soundness and Completeness
3.6 Example: Factorial
3.7 Decorated Programs
Informal Rules — • Decorating a Program —
3.8 Weakest Preconditions

Chapter 4 **Separation Logic** Page

4.1 IMP with Heaps
4.2 Big-Step Operational Semantics
4.3 The Rule of Constancy
4.4 Heap Assertions
4.5 Valid Triples
4.6 The Frame Rule

4.7 Small Axioms for Heap Commands
4.8 Example: Swap
4.9 Linked Lists
4.10 Example: Disposing a List
4.11 List Segments
4.12 Example: List Append
4.13 The Magic Wand

Chapter 1

Operational Semantics

Definition 1.0.1: Multi-step evaluation

Binary relation indicated with $\rightarrow^*$ which is the reflexive and transitive closure of the small-step evaluation.

Note that the transitive rule is a single step and a multi step, although it can be proved that it allows concatenation of single stepps.

1.1 Properties of programs

- Determinism: suppose that for any given configuration, if this can step in different ways these are equal
- Termination: evaluation terminates. Formally, for the multi step there is a configuration that it can multi step to without other possible steps.
- Soundness: if we run a program long enough we should get a final integer, but it's not ture because stores are partial function and if a variable isn't defined we're cooked.

Definition 1.1.1: Free Variables

Given the function $fvs(e)$ defined inductively as:

$$fvs(e) := \begin{cases} \emptyset & \text{if } e \text{ is a variable} \\ fvs(e_1) \cup \{x\} & \text{if } e = (\lambda x.e_1) \\ fvs(e_1) \cup fvs(e_2) & \text{if } e = (e_1 e_2) \end{cases}$$

Note that in $x := x + 1; y := x$ we still need to add x to the free variables because it's in the rhs of the first assignment.

Definition 1.1.2: Well-Formedness

For any environment σ and expression e, if all free variables in e are in σ, then e is well-formed under σ.

Two properties that imply soundness:

Definition 1.1.3: Progress

Every well-formed configuration is either an `Int` or can step to another configuration.

Definition 1.1.4: Preservation

If a configuration is well-formed, any following configuration it stepps to will also be well-formed.

with these lenmas and termination we can show that a well-formaed configuration is sound.

1.1.1 Inductive Sets

A bit of maths before the proof

Definition 1.1.5: Inductive Set

A set S is **inductive** if:

1. $\emptyset \in S$, and
2. for every set x, if $x \in S$, then $x \cup \{x\} \in S$.

We have used inductive setts with the inductive rules (remember that $\longrightarrow$ is a relation, so a set of tuples) and BNF grammar, free-variables and multistep. Another example is the natural numbers using the successor definition.

1.1.2 Inductive Proofs

If we want to prove something about an inductive set (that can be infinite by closing under the inductive rules) we can use the induction principle.
The base cases are all the axioms.
Sometimes the properties are defined over multiple inductive setts, so we need to chose one.

1.2 Large-step Operational Semantics

In the last lecture we defined a semantics for our language of arithmetic expressions using a small-step evaluation relation $\rightarrow \subseteq \textbf{Config} \times \textbf{Config}$ (and its reflexive and transitive closure $\rightarrow^*$). In this lecture we will explore an alternative approach—*large-step* operational semantics—which yields the final result of evaluating an expression directly.

Defining a large-step semantics boils down to specifying a relation $\Downarrow$ that captures the evaluation of an expression. The $\Downarrow$ relation has the following type:

$$\Downarrow \subseteq (\textbf{Store} \times \textbf{Exp}) \times (\textbf{Store} \times \textbf{Int}).$$

We write $\langle \sigma, e \rangle \Downarrow \langle \sigma', n \rangle$ to indicate that $((\sigma, e), (\sigma', n)) \in \Downarrow$. In other words, the expression e with store σ evaluates in one big step to the final store σ' and integer n.

We define the relation $\Downarrow$ inductively, using inference rules:

$$\frac{}{\langle \sigma, n \rangle \Downarrow \langle \sigma, n \rangle}\ \text{INT} \qquad \frac{n = \sigma(x)}{\langle \sigma, x \rangle \Downarrow \langle \sigma, n \rangle}\ \text{VAR}$$

Note:

Note that in the big step semantics, we need an inference rule for every valid configuration, hence the INT rule.

$$\frac{\langle \sigma, e_1 \rangle \Downarrow \langle \sigma', n_1 \rangle \qquad \langle \sigma', e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle \qquad n = n_1 + n_2}{\langle \sigma, e_1 + e_2 \rangle \Downarrow \langle \sigma'', n \rangle}\ \text{ADD}$$

$$\frac{\langle \sigma, e_1 \rangle \Downarrow \langle \sigma', n_1 \rangle \qquad \langle \sigma', e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle \qquad n = n_1 \times n_2}{\langle \sigma, e_1 * e_2 \rangle \Downarrow \langle \sigma'', n \rangle}\ \text{MUL}$$

$$\frac{\langle \sigma, e_1 \rangle \Downarrow \langle \sigma', n_1 \rangle \qquad \langle \sigma'[x \mapsto n_1], e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle}{\langle \sigma, x := e_1; e_2 \rangle \Downarrow \langle \sigma'', n_2 \rangle}\ \text{ASSGN}$$

Example 1.2.1 (Big-step proof)

To illustrate the use of these rules, consider the following proof tree, which shows that evaluating $\langle \sigma, foo := 3; foo * bar \rangle$ using a store σ such that $\sigma(bar) = 7$ yields $\sigma' = \sigma[foo \mapsto 3]$ and 21 as a result:

$$\dfrac{\dfrac{}{\langle \sigma, 3 \rangle \Downarrow \langle \sigma, 3 \rangle}\ \text{INT} \qquad \dfrac{\dfrac{}{\langle \sigma', foo \rangle \Downarrow \langle \sigma', 3 \rangle}\ \text{VAR} \qquad \dfrac{}{\langle \sigma', bar \rangle \Downarrow \langle \sigma', 7 \rangle}\ \text{VAR}}{\langle \sigma', foo * bar \rangle \Downarrow \langle \sigma', 21 \rangle}\ \text{MUL}}{\langle \sigma, foo := 3; foo * bar \rangle \Downarrow \langle \sigma', 21 \rangle}\ \text{ASSGN}$$

A closer look to this structure reveals the relation between small step and large-step evaluation: a depth-first traversal of the large-step proof tree yields the sequence of one-step transitions in small-step evaluation.

Note:

Big step semantics don't capture the computational intricacys (if not well-formed it's undefined instead of blocking), and it strugles with concurrency. For this reason it can be used toghether with small step for the critical parts.

Note:

Also, small step semantics can be used to precysely count computation ticks (cslib 4 lean) for tighter complexity bounds.

1.3 Equivalence of semantics

A natural question to ask is whether the small-step and large-step semantics are equivalent. The next theorem answers this question affirmatively.

Theorem 1.3.1 Equivalence of semantics

For all expressions e, stores σ and σ', and integers n we have:

$$\langle \sigma, e\rangle \Downarrow \langle \sigma', n\rangle \quad \text{if and only if} \quad \langle \sigma, e\rangle \to^* \langle \sigma', n\rangle$$

To streamline the proof, we will work with the following definition of the multi-step relation:

$$\frac{}{\langle \sigma, e\rangle \to^* \langle \sigma, e\rangle}\ \text{REFL} \qquad \frac{\langle \sigma, e\rangle \to \langle \sigma', e'\rangle \qquad \langle \sigma', e'\rangle \to^* \langle \sigma'', e''\rangle}{\langle \sigma, e\rangle \to^* \langle \sigma'', e''\rangle}\ \text{TRANS}$$

Proof Sketch: We show each direction separately.

$\Longrightarrow$: We want to prove that the following property P holds for all expressions $e \in$ **Exp**:

$$P(e) \triangleq \forall \sigma, \sigma' \in \textbf{Store}.\ \forall n \in \textbf{Int}.\ \langle \sigma, e\rangle \Downarrow \langle \sigma', n\rangle \implies \langle \sigma, e\rangle \to^* \langle \sigma', n\rangle$$

We proceed by structural induction on e. We have to consider each of the possible axioms and inference rules for constructing an expression.

Case $e = x$: Assume that $\langle \sigma, x\rangle \Downarrow \langle \sigma', n\rangle$. That is, there is some derivation in the large-step operational semantics whose conclusion is $\langle \sigma, x\rangle \Downarrow \langle \sigma, n\rangle$. There is only one rule whose conclusion matches the configuration $\langle \sigma, x\rangle$: the large-step rule VAR. Thus, we have $n = \sigma(x)$ and $\sigma' = \sigma$. By the small-step rule VAR, we also have $\langle \sigma, x\rangle \to \langle \sigma, n\rangle$. By the REFL and TRANS rules, we conclude that $\langle \sigma, x\rangle \to^* \langle \sigma, n\rangle$, which finishes the case.

Case $e = n$: Assume that $\langle \sigma, n\rangle \Downarrow \langle \sigma', n'\rangle$. There is only one rule whose conclusion matches $\langle \sigma, n\rangle$: the large-step rule INT. Thus, we have $n' = n$ and $\sigma' = \sigma$ and so $\langle \sigma, n\rangle \to^* \langle \sigma, n\rangle$ by the REFL rule.

Case $e = e_1 + e_2$: This is an inductive case. We want to prove that if $P(e_1)$ and $P(e_2)$ hold, then $P(e)$ also holds. Let's write out $P(e_1)$, $P(e_2)$, and $P(e)$ explicitly.

$$P(e_1) = \forall n, \sigma, \sigma'.\ \langle \sigma, e_1\rangle \Downarrow \langle \sigma', n\rangle \implies \langle \sigma, e_1\rangle \to^* \langle \sigma', n\rangle$$
$$P(e_2) = \forall n, \sigma, \sigma'.\ \langle \sigma, e_2\rangle \Downarrow \langle \sigma', n\rangle \implies \langle \sigma, e_2\rangle \to^* \langle \sigma', n\rangle$$
$$P(e) = \forall n, \sigma, \sigma'.\ \langle \sigma, e_1 + e_2\rangle \Downarrow \langle \sigma', n\rangle \implies \langle \sigma, e_1 + e_2\rangle \to^* \langle \sigma', n\rangle$$

Assume that $P(e_1)$ and $P(e_2)$ hold. Also assume that there exist σ, σ' and n such that $\langle \sigma, e_1 + e_2\rangle \Downarrow \langle \sigma', n\rangle$. We need to show that $\langle \sigma, e_1 + e_2\rangle \to^* \langle \sigma', n\rangle$.
We assumed that $\langle \sigma, e_1 + e_2\rangle \Downarrow \langle \sigma', n\rangle$. This means that there is some derivation whose conclusion is $\langle \sigma, e_1 + e_2\rangle \Downarrow \langle \sigma', n\rangle$. By inspection, we see that only one rule has a conclusion of this form: the ADD rule. Thus, the last rule used in the derivation was ADD and it must be the case that $\langle \sigma, e_1\rangle \Downarrow \langle \sigma'', n_1\rangle$ and $\langle \sigma'', e_2\rangle \Downarrow \langle \sigma', n_2\rangle$ hold for some n_1 and n_2 with $n = n_1 + n_2$.
By the induction hypothesis $P(e_1)$, as $\langle \sigma, e_1\rangle \Downarrow \langle \sigma'', n_1\rangle$, we must have $\langle \sigma, e_1\rangle \to^* \langle \sigma'', n_1\rangle$. Likewise, by induction hypothesis $P(e_2)$, we have $\langle \sigma'', e_2\rangle \to^* \langle \sigma', n_2\rangle$. By Lemma 1 below, we have,

$$\langle \sigma, e_1 + e_2\rangle \to^* \langle \sigma'', n_1 + e_2\rangle,$$

and by another application of Lemma 1 we have:

$$\langle \sigma'', n_1 + e_2\rangle \to^* \langle \sigma', n_1 + n_2\rangle$$

Then, using the small-step ADD rule and the multi-step TRANS rule, we have:

$$\frac{\dfrac{n = n_1 + n_2}{\langle \sigma', n_1 + n_2\rangle \to \langle \sigma', n\rangle}\ \text{ADD} \qquad \dfrac{}{\langle \sigma', n\rangle \to^* \langle \sigma', n\rangle}\ \text{REFL}}{\langle \sigma', n_1 + n_2\rangle \to^* \langle \sigma', n\rangle}\ \text{TRANS}$$

Finally, by two applications of Lemma 2, we obtain $\langle \sigma, e_1 + e_2\rangle \to^* \langle \sigma', n\rangle$, which finishes the case.

Case $e = e_1 * e_2$. Similar to case for $e_1 + e_2$ above.

Case $e = x := e_1; e_2$. Omitted. Try it as an exercise.

$\Longleftarrow$: We proceed by induction on the derivation of $\langle \sigma, e\rangle \to^* \langle \sigma', n\rangle$ with a case analysis on the last rule used.

Case Refl: Then $e = n$ and $\sigma' = \sigma$. We immediately have $\langle \sigma, n\rangle \Downarrow \langle \sigma, n\rangle$ by the large-step rule INT.

Case Trans: Then $\langle \sigma, e\rangle \to \langle \sigma'', e''\rangle$ and $\langle \sigma'', e''\rangle \to^* \langle \sigma', n\rangle$. In this case, the induction hypothesis gives $\langle \sigma'', e''\rangle \Downarrow \langle \sigma', n\rangle$. The result follows from Lemma 3 below.

Lenma 1.3.1

If $\langle \sigma, e \rangle \to^* \langle \sigma', n \rangle$, then the following hold:

- $\langle \sigma, e + e_2 \rangle \to^* \langle \sigma', n + e_2 \rangle$
- $\langle \sigma, e * e_2 \rangle \to^* \langle \sigma', n * e_2 \rangle$
- $\langle \sigma, n_1 + e \rangle \to^* \langle \sigma', n_1 + n \rangle$
- $\langle \sigma, n_1 * e \rangle \to^* \langle \sigma', n_1 * n \rangle$
- $\langle \sigma, x := e; e_2 \rangle \to^* \langle \sigma', x := n; e_2 \rangle$

: Omitted; try it as an exercise.

Lenma 1.3.2

If $\langle \sigma, e \rangle \to^* \langle \sigma', e' \rangle$ and $\langle \sigma', e' \rangle \to^* \langle \sigma'', e'' \rangle$, then $\langle \sigma, e \rangle \to^* \langle \sigma'', e'' \rangle$.

Lenma 1.3.3

Omitted; try it as an exercise.

Lenma 1.3.4

If $\langle \sigma, e \rangle \to \langle \sigma'', e'' \rangle$ and $\langle \sigma'', e'' \rangle \Downarrow \langle \sigma', n \rangle$, then $\langle \sigma, e \rangle \Downarrow \langle \sigma', n \rangle$.

Proof. Omitted; try it as an exercise.

1.4 A simple imperative language IMP

We will now consider a more realistic programming language, one where we can assign values to variables and execute control constructs such as **if** and **while**. The syntax for this imperative language, called IMP, is as follows:

$$\begin{array}{rl} \text{arithmetic expressions} & a \in \mathbf{Aexp} \quad a ::= x \mid n \mid a_1 + a_2 \mid a_1 \times a_2 \\ \text{Boolean expressions} & b \in \mathbf{Bexp} \quad b ::= \mathbf{true} \mid \mathbf{false} \mid a_1 < a_2 \\ \text{commands} & c \in \mathbf{Com} \quad c ::= \mathbf{skip} \mid x := a \mid c_1; c_2 \mid \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2 \mid \mathbf{while}\ b\ \mathbf{do}\ c \end{array}$$

1.4.1 Small-step operational semantics

We'll first give a small-step operational semantics for IMP. The configurations in this language are of the form $\langle \sigma, c\rangle$, $\langle \sigma, b\rangle$, and $\langle \sigma, a\rangle$, where σ is a store. The final configurations are of the form $\langle \sigma, \mathbf{skip}\rangle$ for commands, $\langle \sigma, \mathbf{true}\rangle$ and $\langle \sigma, \mathbf{false}\rangle$ for Boolean expressions, and $\langle \sigma, n\rangle$ for arithmetic expressions. There are three different small-step operational semantics relations: one each of the syntactic categories.

$$\rightarrow_{\mathbf{Com}} \subseteq (\mathbf{Store} \times \mathbf{Com}) \times (\mathbf{Store} \times \mathbf{Com})$$

$$\rightarrow_{\mathbf{Bexp}} \subseteq (\mathbf{Store} \times \mathbf{Bexp}) \times (\mathbf{Store} \times \mathbf{Bexp})$$

$$\rightarrow_{\mathbf{Aexp}} \subseteq (\mathbf{Store} \times \mathbf{Aexp}) \times (\mathbf{Store} \times \mathbf{Aexp})$$

For brevity, we will overload the symbol $\rightarrow$ and use it to refer to all of these relations. Which relation is being used will be clear from context. The evaluation rules for arithmetic and Boolean expressions are similar to the ones we've seen before. However, note that since the arithmetic expressions no longer contain assignment, arithmetic and Boolean expressions can not update the store. But to streamline everything we keep it as an output. In the small-step we'll fix the fact that the store doesn't change, but this can be altered if needed using σ'.

Arithmetic expressions

$$\frac{n = \sigma(x)}{\langle \sigma, x\rangle \rightarrow \langle \sigma, n\rangle}$$

$$\frac{\langle \sigma, a_1\rangle \rightarrow \langle \sigma, a_1'\rangle}{\langle \sigma, a_1 + a_2\rangle \rightarrow \langle \sigma, a_1' + a_2\rangle} \qquad \frac{\langle \sigma, a_2\rangle \rightarrow \langle \sigma, a_2'\rangle}{\langle \sigma, n + a_2\rangle \rightarrow \langle \sigma, n + a_2'\rangle} \qquad \frac{p = n + m}{\langle \sigma, n + m\rangle \rightarrow \langle \sigma, p\rangle}$$

$$\frac{\langle \sigma, a_1\rangle \rightarrow \langle \sigma, a_1'\rangle}{\langle \sigma, a_1 \times a_2\rangle \rightarrow \langle \sigma, a_1' \times a_2\rangle} \qquad \frac{\langle \sigma, a_2\rangle \rightarrow \langle \sigma, a_2'\rangle}{\langle \sigma, n \times a_2\rangle \rightarrow \langle \sigma, n \times a_2'\rangle} \qquad \frac{p = n \times m}{\langle \sigma, n \times m\rangle \rightarrow \langle \sigma, p\rangle}$$

Boolean expressions

$$\frac{\langle \sigma, a_1\rangle \rightarrow \langle \sigma, a_1'\rangle}{\langle \sigma, a_1 < a_2\rangle \rightarrow \langle \sigma, a_1' < a_2\rangle} \qquad \frac{\langle \sigma, a_2\rangle \rightarrow \langle \sigma, a_2'\rangle}{\langle \sigma, n < a_2\rangle \rightarrow \langle \sigma, n < a_2'\rangle}$$

$$\frac{n < m}{\langle \sigma, n < m\rangle \rightarrow \langle \sigma, \mathbf{true}\rangle} \qquad \frac{n \geqslant m}{\langle \sigma, n < m\rangle \rightarrow \langle \sigma, \mathbf{false}\rangle}$$

Commands

$$\frac{\langle \sigma, a\rangle \rightarrow \langle \sigma, a'\rangle}{\langle \sigma, x := a\rangle \rightarrow \langle \sigma, x := a'\rangle} \qquad \frac{}{\langle \sigma, x := n\rangle \rightarrow \langle \sigma[x \mapsto n], \mathbf{skip}\rangle}$$

$$\frac{\langle \sigma, c_1\rangle \rightarrow \langle \sigma', c_1'\rangle}{\langle \sigma, c_1; c_2\rangle \rightarrow \langle \sigma', c_1'; c_2\rangle} \qquad \frac{}{\langle \sigma, \mathbf{skip}; c_2\rangle \rightarrow \langle \sigma, c_2\rangle}$$

For **if** commands, we reduce the test until we get **true** or **false** and then we execute the appropriate branch:

$$\frac{\langle \sigma, b\rangle \rightarrow \langle \sigma, b'\rangle}{\langle \sigma, \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2\rangle \rightarrow \langle \sigma, \mathbf{if}\ b'\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2\rangle}$$

$$\frac{}{\langle \sigma, \textbf{if true then } c_1 \textbf{ else } c_2\rangle \to \langle \sigma, c_1\rangle} \qquad \frac{}{\langle \sigma, \textbf{if false then } c_1 \textbf{ else } c_2\rangle \to \langle \sigma, c_2\rangle}$$

For **while** loops, the above strategy doesn't work, because if we evaluate b we won't be able to re-evaluate it for the next loop. Instead, we use the following rule, which can be thought of as "unrolling" the loop, one iteration at a time.

$$\frac{}{\langle \sigma, \textbf{while } b \textbf{ do } c\rangle \to \langle \sigma, \textbf{if } b \textbf{ then } (c; \textbf{while } b \textbf{ do } c) \textbf{ else skip}\rangle}$$

Note:

With this new rule, induction on the structure of the term doesn't work! This is because instead of always getting smaller, the step is to a bigger c. Sometimes this is managable, but not always.

We can now take a concrete program and see how it executes under the above rules. Consider we execute the program

Example 1.4.1 (Small-step eval of IMP)

$$foo := 3;\ \textbf{while } foo < 4 \textbf{ do } foo := foo + 5$$

The execution works as follows:

$$\begin{aligned}
&\langle \sigma, foo := 3; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma', \textbf{skip}; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle && \text{where } \sigma' = \sigma[foo \mapsto 3] \\
\to\ &\langle \sigma', \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma', \textbf{if } foo < 4 \textbf{ then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma', \textbf{if } 3 < 4 \textbf{ then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma', \textbf{if true then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma', foo := foo + 5; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma', foo := 3 + 5; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma', foo := 8; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma'', \textbf{skip}; \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle && \text{where } \sigma'' = \sigma'[foo \mapsto 8] \\
\to\ &\langle \sigma'', \textbf{while } foo < 4 \textbf{ do } foo := foo + 5\rangle \\
\to\ &\langle \sigma'', \textbf{if } foo < 4 \textbf{ then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma'', \textbf{if } 8 < 4 \textbf{ then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma'', \textbf{if false then } (foo := foo + 5; W) \textbf{ else skip}\rangle \\
\to\ &\langle \sigma'', \textbf{skip}\rangle
\end{aligned}$$

where W is an abbreviation for the while loop **while** $foo < 4$ **do** $foo := foo + 5$.

1.5 Large-step operational semantics for IMP

We define large-step evaluation relations for arithmetic expressions, Boolean expressions, and commands. The relation for arithmetic expressions relates an arithmetic expression and store to the integer value that the expression evaluates to. For Boolean expressions, the final value is in **Bool** = {**true**, **false**}. For commands, the final value is a store.

$$\Downarrow_{\textbf{Aexp}} \subseteq (\textbf{Aexp} \times \textbf{Store}) \times \textbf{Int}$$

$$\Downarrow_{\textbf{Bexp}} \subseteq (\textbf{Bexp} \times \textbf{Store}) \times \textbf{Bool}$$

$$\Downarrow_{\textbf{Com}} \subseteq (\textbf{Com} \times \textbf{Store}) \times \textbf{Store}$$

Again, we overload the symbol $\Downarrow$ and use it for any of these three relations; which relation is intended will be clear from context. We also use infix notation, for example writing $\langle \sigma, c \rangle \Downarrow \sigma'$ if $(\langle \sigma, c \rangle, \sigma') \in \Downarrow_{\mathbf{Com}}$.

Note:

In this course, we're not interested in concrete syntax, so ambiguous AST aren't a problem. Braces can be used to explicit.

Arithmetic expressions.

$$\frac{}{\langle \sigma, n \rangle \Downarrow n} \qquad \frac{\sigma(x) = n}{\langle \sigma, x \rangle \Downarrow n}$$

$$\frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \qquad \langle \sigma, a_2 \rangle \Downarrow n_2 \qquad n = n_1 + n_2}{\langle \sigma, a_1 + a_2 \rangle \Downarrow n} \qquad \frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \qquad \langle \sigma, a_2 \rangle \Downarrow n_2 \qquad n = n_1 \times n_2}{\langle \sigma, a_1 \times a_2 \rangle \Downarrow n}$$

Boolean expressions.

$$\frac{}{\langle \sigma, \mathbf{true} \rangle \Downarrow \mathbf{true}} \qquad \frac{}{\langle \sigma, \mathbf{false} \rangle \Downarrow \mathbf{false}}$$

$$\frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \qquad \langle \sigma, a_2 \rangle \Downarrow n_2 \qquad n_1 < n_2}{\langle \sigma, a_1 < a_2 \rangle \Downarrow \mathbf{true}} \qquad \frac{\langle \sigma, a_1 \rangle \Downarrow n_1 \qquad \langle \sigma, a_2 \rangle \Downarrow n_2 \qquad n_1 \geqslant n_2}{\langle \sigma, a_1 < a_2 \rangle \Downarrow \mathbf{false}}$$

Commands.

$$\frac{}{\langle \sigma, \mathbf{skip} \rangle \Downarrow \sigma} \text{ SKIP} \qquad \frac{\langle \sigma, a \rangle \Downarrow n}{\langle \sigma, x := a \rangle \Downarrow \sigma[x \mapsto n]} \text{ ASSGN}$$

$$\frac{\langle \sigma, c_1 \rangle \Downarrow \sigma' \qquad \langle \sigma', c_2 \rangle \Downarrow \sigma''}{\langle \sigma, c_1; c_2 \rangle \Downarrow \sigma''} \text{ SEQ}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \mathbf{true} \qquad \langle \sigma, c_1 \rangle \Downarrow \sigma'}{\langle \sigma, \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2 \rangle \Downarrow \sigma'} \text{ IF-T} \qquad \frac{\langle \sigma, b \rangle \Downarrow \mathbf{false} \qquad \langle \sigma, c_2 \rangle \Downarrow \sigma'}{\langle \sigma, \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2 \rangle \Downarrow \sigma'} \text{ IF-F}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \mathbf{false}}{\langle \sigma, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \sigma} \text{ WHILE-F}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \mathbf{true} \qquad \langle \sigma, c \rangle \Downarrow \sigma' \qquad \langle \sigma', \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \sigma''}{\langle \sigma, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \sigma''} \text{ WHILE-T}$$

Note:

The big-step semantics treat termination differently to small-step. To prove these two are equal, we could assume termination.

It's interesting to see that the rule for while loops does not rely on using an if command (as in the case of small-step semantics). Why does this rule work?

1.5.1 Command equivalence

The small-step operational semantics suggests that the loop **while** b **do** c should be equivalent to the command **if** b **then** $(c;$ **while** b **do** $c)$ **else skip**. Can we show that this indeed the case that the language is defined using the above large-step evaluation?

First, we need to to be more precise about what "equivalent commands" mean. Our formal model allows us to define this concept using large-step evaluations as follows. (One can write a similar definition using $\rightarrow^*$ in small-step semantics.)

Definition 1.5.1: Equivalence of commands

Two commands c and c' are equivalent (written $c \sim c'$) if, for any stores σ and σ', we have

$$\langle \sigma, c \rangle \Downarrow \sigma' \iff \langle \sigma, c' \rangle \Downarrow \sigma'.$$

We can now state and prove the claim that **while** b **do** c and **if** b **then** $(c;$ **while** b **do** $c)$ **else skip** are equivalent.

Theorem 1.5.1

For all $b \in$ **Bexp** and $c \in$ **Com** we have

$$\textbf{while } b \textbf{ do } c \ \sim \ \textbf{if } b \textbf{ then } (c; \textbf{while } b \textbf{ do } c) \textbf{ else skip}.$$

: Let W be an abbreviation for **while** b **do** c. We want to show that for all stores σ, σ', we have:

$$\langle \sigma, W \rangle \Downarrow \sigma' \quad \text{if and only if} \quad \langle \sigma, \textbf{if } b \textbf{ then } (c; W) \textbf{ else skip} \rangle \Downarrow \sigma'$$

For this, we must show that both directions ($\Longrightarrow$ and $\Longleftarrow$) hold. We'll show only direction $\Longrightarrow$; the other is similar.
Assume that σ and σ' are stores such that $\langle \sigma, W \rangle \Downarrow \sigma'$. It means that there is some derivation that proves this fact. Inspecting the evaluation rules, we see that there are two possible rules whose conclusions match this fact: While-F and While-T. We analyze each of them in turn.

While-F. The derivation must look like the following.

$$\frac{\begin{array}{c}\vdots_1 \\ \langle \sigma, b \rangle \Downarrow \textbf{false}\end{array}}{\langle \sigma, W \rangle \Downarrow \sigma} \text{ WHILE-F}$$

Here, we use $\vdots_1$ to refer to the derivation of $\langle \sigma, b \rangle \Downarrow \textbf{false}$. Note that in this case, $\sigma' = \sigma$.

We can use $\vdots_1$ to derive a proof tree showing that the evaluation of **if** b **then** $(c; W)$ **else skip** yields the same final state σ:

$$\frac{\begin{array}{c}\vdots_1 \\ \langle \sigma, b \rangle \Downarrow \textbf{false}\end{array} \qquad \frac{}{\langle \sigma, \textbf{skip} \rangle \Downarrow \sigma} \text{ SKIP}}{\langle \sigma, \textbf{if } b \textbf{ then } (c; W) \textbf{ else skip} \rangle \Downarrow \sigma} \text{ IF-F}$$

While-T. In this case, the derivation has the following form.

$$\frac{\begin{array}{c}\vdots_2 \\ \langle \sigma, b \rangle \Downarrow \textbf{true}\end{array} \qquad \begin{array}{c}\vdots_3 \\ \langle \sigma, c \rangle \Downarrow \sigma''\end{array} \qquad \begin{array}{c}\vdots_4 \\ \langle \sigma'', W \rangle \Downarrow \sigma'\end{array}}{\langle \sigma, W \rangle \Downarrow \sigma'} \text{ WHILE-T}$$

We can use subderivations $\vdots_2$, $\vdots_3$, and $\vdots_4$ to show that the evaluation of **if** b **then** $(c; W)$ **else skip** yields the same final state σ'.

$$\frac{\begin{array}{c}\vdots_2 \\ \langle \sigma, b \rangle \Downarrow \textbf{true}\end{array} \qquad \frac{\begin{array}{c}\vdots_3 \\ \langle \sigma, c \rangle \Downarrow \sigma''\end{array} \qquad \begin{array}{c}\vdots_4 \\ \langle \sigma'', W \rangle \Downarrow \sigma'\end{array}}{\langle \sigma, c; W \rangle \Downarrow \sigma'} \text{ SEQ}}{\langle \sigma, \textbf{if } b \textbf{ then } (c; W) \textbf{ else skip} \rangle \Downarrow \sigma'} \text{ IF-T}$$

Hence, we showed that in each of the two possible cases, the command **if** b **then** $(c; W)$ **else skip** evaluates to the same final state as the command W. ☺

Some questions:

- Can we write a program that doesn't terminate? yes, obv
- Is IMP Turing Complete? Kinda, we don't have a way of creating new state (it's statically allocated), so if we replace the unbound **Int** type with something like **Int64**, it wouldn't be complete

Chapter 2

Denotational Semantics

We have now seen two operational models for programming languages: small-step and large-step. In this lecture, we consider a different semantic model, called *denotational semantics*.
The idea in denotational semantics is to express the meaning of a program as the mathematical function that expresses what the program computes. We can think of an IMP program c as a function from stores to stores: given an initial store, the program produces a final store. For example, the program $foo := bar + 1$ can be thought of as a function that when given an input store σ, produces a final store σ' that is identical to σ except that it maps foo to the integer $\sigma(bar) + 1$; that is, $\sigma' = \sigma[foo \mapsto \sigma(bar) + 1]$. We will model programs as functions from input stores to output stores. As opposed to operational models, which tell us *how* programs execute, the denotational model shows us *what* programs compute.

2.1 A Denotational Semantics for IMP

For each program c, we write $C[\![c]\!]$ for the *denotation* of c, that is, the mathematical function that c represents:

$$C[\![c]\!] : \textbf{Store} \rightharpoonup \textbf{Store}.$$

Note that $C[\![c]\!]$ is actually a partial function (as opposed to a total function), both because the store may not be defined on the free variables of the program and because the program may not terminate for certain input stores. The function $C[\![c]\!]$ is not defined for non-terminating programs as they have no corresponding output stores.
We will write $C[\![c]\!]\sigma$ for the result of applying the function $C[\![c]\!]$ to the store σ. That is, if f is the function that $C[\![c]\!]$ denotes, then we write $C[\![c]\!]\sigma$ to mean the same thing as $f(\sigma)$.
We must also model expressions as functions, this time from stores to the values they represent. We will write $\mathcal{A}[\![a]\!]$ for the denotation of arithmetic expression a, and $\mathcal{B}[\![b]\!]$ for the denotation of boolean expression b.

$$\mathcal{A}[\![a]\!] : \textbf{Store} \rightharpoonup \textbf{Int} \qquad \mathcal{B}[\![b]\!] : \textbf{Store} \rightharpoonup \{\textbf{true}, \textbf{false}\}$$

Now we want to define these functions. To make it easier to write down these definitions, we will describe (partial) functions using sets of pairs. More precisely, we will represent a partial map $f : A \rightharpoonup B$ as a set of pairs $F = \{(a, b) \mid a \in A \text{ and } b = f(a) \in B\}$ such that, for each $a \in A$, there is at most one pair of the form $(a, _)$ in the set. Hence $(a, b) \in F$ is the same as $b = f(a)$.
We can now define denotations for IMP. We start with the denotations of expressions:

$$\begin{aligned}
\mathcal{A}[\![n]\!] &= \{(\sigma, n)\} \\
\mathcal{A}[\![x]\!] &= \{(\sigma, \sigma(x))\} \\
\mathcal{A}[\![a_1 + a_2]\!] &= \{(\sigma, n) \mid (\sigma, n_1) \in \mathcal{A}[\![a_1]\!] \wedge (\sigma, n_2) \in \mathcal{A}[\![a_2]\!] \wedge n = n_1 + n_2\}
\end{aligned}$$

$$\begin{aligned}
\mathcal{B}[\![\textbf{true}]\!] &= \{(\sigma, \textbf{true})\} \\
\mathcal{B}[\![\textbf{false}]\!] &= \{(\sigma, \textbf{false})\} \\
\mathcal{B}[\![a_1 < a_2]\!] &= \{(\sigma, \textbf{true}) \mid (\sigma, n_1) \in \mathcal{A}[\![a_1]\!] \wedge (\sigma, n_2) \in \mathcal{A}[\![a_2]\!] \wedge n_1 < n_2\} \\
&\quad \cup \{(\sigma, \textbf{false}) \mid (\sigma, n_1) \in \mathcal{A}[\![a_1]\!] \wedge (\sigma, n_2) \in \mathcal{A}[\![a_2]\!] \wedge n_1 \geqslant n_2\}
\end{aligned}$$

The denotations for commands are as follows:

$$\begin{aligned} C[\![\textbf{skip}]\!] &= \{(\sigma, \sigma)\} \\ C[\![x := a]\!] &= \{(\sigma, \sigma[x \mapsto n]) \mid (\sigma, n) \in \mathcal{A}[\![a]\!]\} \\ C[\![c_1; c_2]\!] &= \{(\sigma, \sigma') \mid \exists \sigma''.\ ((\sigma, \sigma'') \in C[\![c_1]\!] \wedge (\sigma'', \sigma') \in C[\![c_2]\!])\} \end{aligned}$$

All these can be expressed in function-style notation too.

Note:

If the denotation of σ under b is undefined, then the function should also be undefined, hence the case completeness.

Note that $C[\![c_1; c_2]\!] = C[\![c_2]\!] \circ C[\![c_1]\!]$, where $\circ$ is the composition of relations, defined as follows: if $R_1 \subseteq A \times B$ and $R_2 \subseteq B \times C$ then $R_2 \circ R_1 \subseteq A \times C$ is $R_2 \circ R_1 = \{(a, c) \mid \exists b \in B.\ (a, b) \in R_1 \wedge (b, c) \in R_2\}$. If $C[\![c_1]\!]$ and $C[\![c_2]\!]$ are total functions, then $\circ$ is function composition.

$$\begin{aligned} C[\![\textbf{if } b \textbf{ then } c_1 \textbf{ else } c_2]\!] = &\{(\sigma, \sigma') \mid (\sigma, \textbf{true}) \in \mathcal{B}[\![b]\!] \wedge (\sigma, \sigma') \in C[\![c_1]\!]\} \\ &\cup \{(\sigma, \sigma') \mid (\sigma, \textbf{false}) \in \mathcal{B}[\![b]\!] \wedge (\sigma, \sigma') \in C[\![c_2]\!]\} \end{aligned}$$

$$\begin{aligned} C[\![\textbf{while } b \textbf{ do } c]\!] = &\{(\sigma, \sigma) \mid (\sigma, \textbf{false}) \in \mathcal{B}[\![b]\!]\} \\ &\cup \{(\sigma, \sigma') \mid (\sigma, \textbf{true}) \in \mathcal{B}[\![b]\!] \wedge \exists \sigma''.\ ((\sigma, \sigma'') \in C[\![c]\!] \wedge (\sigma'', \sigma') \in C[\![\textbf{while } b \textbf{ do } c]\!])\} \end{aligned}$$

But now we have a problem: the last "definition" is not really a definition because it expresses $C[\![\textbf{while } b \textbf{ do } c]\!]$ in terms of itself! This is not a definition but a recursive equation. What we want is the solution to this equation.

Note:

The second invocation of while isn't completely circular because the state changes.

2.2 Fixed Points

We gave a recursive equation that the function $C[\![\textbf{while } b \textbf{ do } c]\!]$ must satisfy. To understand some of the issues involved, let's consider a simpler example. Consider the following equation for a function $f : \mathbb{N} \to \mathbb{N}$.

$$f(x) = \begin{cases} 0 & \text{if } x = 0 \\ f(x-1) + 2x - 1 & \text{otherwise} \end{cases} \tag{2.1}$$

This is not a definition for f, but rather an equation that we want f to satisfy. What function, or functions, satisfy this equation for f? The only solution to this equation is the function $f(x) = x^2$.
In general, there may be no solutions for a recursive equation (e.g., there are no functions $g : \mathbb{N} \to \mathbb{N}$ that satisfy the recursive equation $g(x) = g(x) + 1$), or multiple solutions (e.g., find two functions $g : \mathbb{R} \to \mathbb{R}$ that satisfy $g(x) = 4 \times g(\frac{x}{2})$).
We can compute solutions to such equations by building successive approximations. Each approximation is closer and closer to the solution. To solve the recursive equation for f, we start with the partial function $f_0 = \emptyset$ (i.e., f_0 is the empty relation; it is a partial function with the empty set for its domain). We compute successive

approximations using the recursive equation.

$$
\begin{aligned}
f_0 &= \emptyset \\
f_1 &= \begin{cases} 0 & \text{if } x = 0 \\ f_0(x-1) + 2x - 1 & \text{otherwise} \end{cases} = \{(0,0)\} \\
f_2 &= \begin{cases} 0 & \text{if } x = 0 \\ f_1(x-1) + 2x - 1 & \text{otherwise} \end{cases} = \{(0,0),(1,1)\} \\
f_3 &= \begin{cases} 0 & \text{if } x = 0 \\ f_2(x-1) + 2x - 1 & \text{otherwise} \end{cases} = \{(0,0),(1,1),(2,4)\} \\
&\vdots
\end{aligned}
$$

This sequence of successive approximations f_i gradually builds the function $f(x) = x^2$.
We can model this process of successive approximations using a higher-order function F that takes one approximation f_k and returns the next approximation f_{k+1}:

$$F : (\mathbb{N} \rightharpoonup \mathbb{N}) \to (\mathbb{N} \rightharpoonup \mathbb{N})$$

where

$$(F(f))(x) = \begin{cases} 0 & \text{if } x = 0 \\ f(x-1) + 2x - 1 & \text{otherwise} \end{cases}$$

A solution to the recursive equation (1) is a function f such that $f = F(f)$. In general, given a function $F : A \to A$, we have that $a \in A$ is a *fixed point* of F if $F(a) = a$. We also write $a = \text{fix}(F)$ to indicate that a is a least fixed point of F.
So the solution to the recursive equation (1) is a fixed-point of the higher-order function F. We can compute this fixed point iteratively, starting with $f_0 = \emptyset$ and at each iteration computing $f_{k+1} = F(f_k)$. The fixed point is the limit of this process:

$$
\begin{aligned}
f &= \text{fix}(F) \\
&= f_0 \cup f_1 \cup f_2 \cup f_3 \cup \ldots \\
&= \emptyset \cup F(\emptyset) \cup F(F(\emptyset)) \cup F(F(F(\emptyset))) \cup \ldots \\
&= \bigcup_{i \geqslant 0} F^i(\emptyset)
\end{aligned}
$$

2.3 Denotation for Loops

We can now write the correct denotation case for while loops as the fixed point of a higher-order function:

$$\mathcal{C}[\![\textbf{while } b \textbf{ do } c]\!] = \text{fix}(F)$$

where

$$F(f) = \{(\sigma,\sigma) \mid (\sigma, \textbf{false}) \in \mathcal{B}[\![b]\!]\} \cup \{(\sigma,\sigma') \mid (\sigma, \textbf{true}) \in \mathcal{B}[\![b]\!] \wedge \exists \sigma''.\ (\sigma,\sigma'') \in \mathcal{C}[\![c]\!] \wedge (\sigma'',\sigma') \in f\}$$

2.4 Kleene's Fixed Point Theorem

Next we'll prove Kleene's fixpoint theorem, which shows that the fixed point used to define the semantics of while commands exists.

Definition 2.4.1: Scott Continuity

A function F from U to U is said to be *Scott-continuous* if for every chain $X_1 \subseteq X_2 \subseteq \ldots$ we have $F(\bigcup_i X_i) = \bigcup_i F(X_i)$.

It is not hard to show that if F is Scott continuous, then it is also *monotonic*—that is, $X \subseteq Y$ implies $F(X) \subseteq F(Y)$. The proof of this fact is left as an exercise.

Theorem 2.4.1 Kleene Fixpoint

Let F be a Scott-continuous function. The least fixed point of F is $\bigcup_i F^i(\emptyset)$.

: Let $X = \bigcup_i F^i(\emptyset)$.
First, we will prove that X is a fixed point of F—that is, $F(X) = X$. We calculate as follows:

$$\begin{aligned}
F(X) &= F\left(\bigcup_i F^i(\emptyset)\right) && \text{By definition of } X \\
&= \bigcup_i F(F^i(\emptyset)) && \text{By Scott continuity} \\
&= \bigcup_i F^{i+1}(\emptyset) \\
&= \emptyset \cup \bigcup_i F^{i+1}(\emptyset) \\
&= F^0(\emptyset) \cup \bigcup_i F^{i+1}(\emptyset) \\
&= \bigcup_i F^i(\emptyset) \\
&= X
\end{aligned}$$

Second, we will show that X is the least fixed point of F. Suppose that Y is some other arbitrary fixed point of F. By induction, we can easily show that $F^i(\emptyset) \subseteq Y$ for all i. For the base case, i is 0 and we trivially have $F^0(\emptyset) = \emptyset \subseteq Y$. For the inductive case, we assume that $F^i(\emptyset) \subseteq Y$ and prove that $F^{i+1}(\emptyset) \subseteq Y$. By our inductive hypothesis and the fact that F is monotone, we have that $F(F^i(\emptyset)) \subseteq F(Y)$. As Y is a fixed point we also have $F(Y) = Y$ and so $F^{i+1}(\emptyset) \subseteq Y$.
Then, since every element of the chain

$$F^0(\emptyset) \subseteq F^1(\emptyset) \subseteq \ldots$$

is a subset of Y, immediately we have that their union, $X = \bigcup_i F^i(\emptyset) \subseteq Y$. Hence, X is the least (with respect to $\subseteq$) fixed point of F.

2.5 Reasoning using Denotational Semantics

One of the main advantages of using denotational semantics compared to operational semantics is that proofs of equivalence can be carried out directly by simply calculating the denotations of programs and then arguing that they are identical. This is in contrast to operational techniques, where one must reason explicitly about low-level transitions and derivations involving ad hoc abstract machines.
As an example, to show that **skip**; c and c; **skip** are equivalent, we can calculate as follows,

$$\begin{aligned}
C[\![\textbf{skip}; c]\!] &= \{(\sigma, \sigma'') \mid \exists \sigma'.\ (\sigma, \sigma') \in C[\![\textbf{skip}]\!] \wedge (\sigma', \sigma'') \in C[\![c]\!]\} \\
&= \{(\sigma, \sigma'') \mid (\sigma, \sigma'') \in C[\![c]\!]\} \\
&= \{(\sigma, \sigma'') \mid \exists \sigma'.\ (\sigma, \sigma') \in C[\![c]\!] \wedge (\sigma', \sigma'') \in C[\![\textbf{skip}]\!]\} \\
&= C[\![c; \textbf{skip}]\!]
\end{aligned}$$

using standard facts about partial functions, relations, and sets as convenient in the proof itself.
Next, consider the command $C[\![\textbf{while false do } c]\!]$. By the definition of the denotational semantics, this is equal to fix(F) where

$$F(f) = \{(\sigma, \sigma) \mid (\sigma, \textbf{false}) \in \mathcal{B}[\![b]\!]\} \cup \{(\sigma, \sigma'') \mid (\sigma, \textbf{true}) \in \mathcal{B}[\![b]\!] \wedge (\sigma, \sigma') \in C[\![c]\!] \wedge (\sigma', \sigma'') \in f\}$$

By the Kleene fixpoint theorem we have that $\text{fix}(F) = \bigcup_i F^i(\emptyset)$. It is straightforward to show by induction that since the guard is false, for all i we have $F^i(\emptyset) = \{(\sigma, \sigma)\}$. It follows that $\text{fix}(F) = \{(\sigma, \sigma)\}$, which is just $C[\![\textbf{skip}]\!]$.
As an exercise, calculate $C[\![\textbf{while true do skip}]\!]$ using the same technique.

Chapter 3

Axiomatic Semantics

3.1 Introduction to axiomatic semantics

Now we turn to the third and final main style of semantics, *axiomatic semantics*. The idea in axiomatic semantics is to define meaning in terms of logical specifications that programs satisfy. This is in contrast to operational models (which show *how* programs execute) and denotational models (which show *what* programs compute). This approach to reasoning about programs and expressing program semantics was originally proposed by Floyd and Hoare and was developed further by Dijkstra and Gries.
A common way to express program specifications is in terms of pre-conditions and post-conditions:

$$\{Pre\}\ c\ \{Post\}$$

where c is a program, and *Pre* and *Post* are formulas that describe properties of the program state, usually referred to as *assertions*. Such a triple is usually referred to as a *partial correctness specification* (or sometimes a "Hoare triple") and has the following meaning:

> "If *Pre* holds before executing c, and c terminates, then *Post* holds after c."

In other words, if we start with a store σ in which *Pre* holds and the execution of c with respect to σ terminates and yields a store σ', then *Post* holds in store σ'.
Pre-conditions and post-conditions can be regarded as interfaces or contracts between the program and its clients. They help users to understand what the program is supposed to yield without needing to understand how the program executes. Typically, programmers write them as comments for procedures and functions as documentation and to make it easier to maintain programs. Such specifications are especially useful for library functions for which the source code is often not available to the users. In this case, pre-conditions and post-conditions serve as contracts between the library developers and users of the library.
However, there is no guarantee that pre-conditions and post-conditions written informally in comments are correct: the comments describe the intent of the developer, but they do not give a guarantee of correctness. Axiomatic semantics addresses this problem. It shows how to rigorously describe partial correctness statements and how to establish correctness using formal reasoning.
Note that partial correctness specifications don't ensure that the program will terminate—this is why they are called "partial". In contrast, *total correctness statements* ensure that the program terminates whenever the precondition holds. Such statements are denoted using square brackets:

$$[\ Pre\ ]\ c\ [\ Post\ ]$$

meaning:

> "If *Pre* holds before c then c will terminate and *Post* will hold after c."

In general a pre-condition specifies what the program expects before execution and the post-condition specifies what guarantees the program provides (if the program terminates). Here is a simple example:

$$\{foo = 0 \wedge bar = i\}\ baz := 0;\ \textbf{while}\ foo \neq bar\ \textbf{do}\ (baz := baz - 2;\ foo := foo + 1)\ \{baz = -2i\}$$

It says that if the store maps *foo* to 0 and *bar* to i before execution, then, if the program terminates, the final store will map *baz* to $-2i$ (i.e., -2 times the initial value of *bar*). Note that i is a *logical variable* that doesn't occur in the program and is only used to express the initial value of *bar*. Such variables are sometimes called *ghost variables*.
This partial correctness statement is valid. That is, it is indeed the case that if we have any store σ such that $\sigma(foo) = 0$, and

$$C[\![baz := 0;\ \textbf{while}\ foo \neq bar\ \textbf{do}\ (baz := baz - 2;\ foo := foo + 1)]\!]\sigma = \sigma',$$

then $\sigma'(baz) = -2\sigma(bar)$.
Note that this is a partial correctness statement: if the pre-condition is true before c, and c terminates then the post-condition holds after c. There are some initial stores for which the program will not terminate.
The following total correctness statement is true.

$$[foo = 0 \wedge bar = i \wedge i \geqslant 0]\ baz := 0;\ \textbf{while}\ foo \neq bar\ \textbf{do}\ (baz := baz - 2;\ foo := foo + 1)\ [baz = -2i]$$

That is, if we start with a store σ that maps *foo* to 0 and *bar* to a non-negative integer, then the execution of the command will terminate in a final store σ' with $\sigma'(baz) = -2\sigma(bar)$.
The following partial correctness statement is not valid. (Why not?)

$$\{foo = 0 \wedge bar = i\}\ baz := 0;\ \textbf{while}\ foo \neq bar\ \textbf{do}\ (baz := baz + foo;\ foo := foo + 1)\ \{baz = i\}$$

In the rest of our discussion of axiomatic semantics we will focus almost exclusively on partial correctness assertions.

3.2 Assertions

Now we turn to the following issues:

- What logic do we use for writing assertions? That is, what can we express in pre-conditions and post-conditions?
- What does it mean that an assertion is valid? What does it mean that a partial correctness statement $\{Pre\}\ c\ \{Post\}$ is valid?
- How can we prove that a partial correctness statement is valid?

What can we say in pre-conditions and post-conditions? In the examples so far, we have used program variables, equality, logical variables (e.g., i), and conjunction ($\wedge$). What we allow in pre-conditions and post-conditions directly influences the sorts of program properties we can describe using partial correctness statements. We will use the set of logical formulas including comparisons between arithmetic expressions, standard logical operators (and, or, implication, negation), as well as quantifiers (universal and existential). Assertions may also introduce logical variables that are different than the variables appearing in the program.

$$\begin{aligned} i, j &\in \textbf{LVar} \\ a \in \textbf{Aexp} &::= x \mid i \mid n \mid a_1 + a_2 \mid a_1 \times a_2 \\ P, Q \in \textbf{Assn} &::= \textbf{true} \mid \textbf{false} \mid a_1 < a_2 \mid P_1 \wedge P_2 \mid P_1 \vee P_2 \mid P_1 \Rightarrow P_2 \mid \neg P \mid \forall i.\ P \mid \exists i.\ P \end{aligned}$$

Observe that the domain of boolean expressions **Bexp** is a subset of the domain of assertions. Notable additions over the syntax of boolean expressions are quantifiers ($\forall$ and $\exists$). For instance, one can express the fact that variable x divides variable y using existential quantification: $\exists i.\ x \times i = y$.

3.3 Satisfaction and Validity

Now we would like to describe what we mean by "assertion P holds in store σ". But to determine whether P holds or not, we need more than just the store σ (which maps program variables to their values); we also need to know the values of the logical variables. We describe those values using an *interpretation* I,

$$I : \textbf{LVar} \to \textbf{Int},$$

and define the function $\mathcal{A}_i[\![a]\!]$, which is like the denotation of expressions extended to logical variables in the obvious way:

$$\begin{aligned}
\mathcal{A}_i[\![n]\!](\sigma, I) &= n \\
\mathcal{A}_i[\![x]\!](\sigma, I) &= \sigma(x) \\
\mathcal{A}_i[\![i]\!](\sigma, I) &= I(i) \\
\mathcal{A}_i[\![a_1 + a_2]\!](\sigma, I) &= \mathcal{A}_i[\![a_1]\!](\sigma, I) + \mathcal{A}_i[\![a_2]\!](\sigma, I)
\end{aligned}$$

Now we can express the satisfiability of assertions as a relation $\sigma \models_I P$ read as "P is satisfied in store σ under interpretation I," or "store σ satisfies assertion P under interpretation I." We will write $\sigma \not\models_I P$ whenever $\sigma \models_I P$ doesn't hold.

$$\begin{aligned}
\sigma \models_I \textbf{true} &\quad \text{(always)} \\
\sigma \models_I a_1 < a_2 &\quad \text{if } \mathcal{A}_i[\![a_1]\!](\sigma, I) < \mathcal{A}_i[\![a_2]\!](\sigma, I) \\
\sigma \models_I P_1 \wedge P_2 &\quad \text{if } \sigma \models_I P_1 \text{ and } \sigma \models_I P_2 \\
\sigma \models_I P_1 \vee P_2 &\quad \text{if } \sigma \models_I P_1 \text{ or } \sigma \models_I P_2 \\
\sigma \models_I P_1 \Rightarrow P_2 &\quad \text{if } \sigma \not\models_I P_1 \text{ or } \sigma \models_I P_2 \\
\sigma \models_I \neg P &\quad \text{if } \sigma \not\models_I P \\
\sigma \models_I \forall i.\ P &\quad \text{if } \forall k \in \mathit{Int}.\ \sigma \models_{I[i \mapsto k]} P \\
\sigma \models_I \exists i.\ P &\quad \text{if } \exists k \in \mathit{Int}.\ \sigma \models_{I[i \mapsto k]} P
\end{aligned}$$

We can now say that an assertion P is *valid* (written $\models P$) if it is valid in any store, under any interpretation: $\forall \sigma, I.\ \sigma \models_I P$.

Having defined validity for individual assertions, we now turn to partial correctness statements. We say that a partial correctness statement $\{P\}\ c\ \{Q\}$ is *satisfied* in store σ and interpretation I, written $\sigma \models_I \{P\}\ c\ \{Q\}$, if:

$$\forall \sigma'.\ \text{if } \sigma \models_I P \text{ and } C[\![c]\!]\sigma = \sigma' \text{ then } \sigma' \models_I Q$$

Note that this definition depends on the execution of c in the initial store σ.

Finally, we can say that a partial correctness triple is *valid* (written $\models \{P\}\ c\ \{Q\}$), if it is valid in any store and interpretation:

$$\forall \sigma, I.\ \sigma \models_I \{P\}\ c\ \{Q\}.$$

Now we know what we mean when we say "assertion P holds" or "partial correctness statement $\{P\}\ c\ \{Q\}$ is valid."

3.4 Hoare Logic

How do we show that a partial correctness statement $\{P\}\ c\ \{Q\}$ holds? We know that $\{P\}\ c\ \{Q\}$ is valid if it holds for all stores and interpretations: $\forall \sigma, I.\ \sigma \models_I \{P\}\ c\ \{Q\}$. Furthermore, showing that $\sigma \models_I \{P\}\ c\ \{Q\}$ requires reasoning about the execution of command c (that is, $C[\![c]\!]$), as indicated by the definition of validity.

It turns out that there is an elegant way of deriving valid partial correctness statements, without having to reason about stores, interpretations, and the execution of c. We can use a set of inference rules and axioms, called *Hoare rules*, to directly derive valid partial correctness statements. The set of rules forms a proof system known as *Hoare logic*.

$$\frac{}{\{P\}\ \textbf{skip}\ \{P\}}\ \text{SKIP} \qquad \frac{}{\{P[a/x]\}\ x := a\ \{P\}}\ \text{ASSIGN}$$

$$\frac{\{P\}\ c_1\ \{R\} \qquad \{R\}\ c_2\ \{Q\}}{\{P\}\ c_1; c_2\ \{Q\}}\ \text{SEQ}$$

$$\frac{\{P \wedge b\}\ c_1\ \{Q\} \qquad \{P \wedge \neg b\}\ c_2\ \{Q\}}{\{P\}\ \textbf{if}\ b\ \textbf{then}\ c_1\ \textbf{else}\ c_2\ \{Q\}}\ \text{IF}$$

$$\frac{\{P \wedge b\}\ c\ \{P\}}{\{P\}\ \textbf{while}\ b\ \textbf{do}\ c\ \{P \wedge \neg b\}}\ \text{WHILE}$$

The assertion P in the rule for while loops is essentially a *loop invariant*; it is an assertion that holds before and after each iteration, as shown in the premise of the rule. Therefore, it is both a pre-condition for the loop (because it holds before the first iteration); and also a post-condition for the loop (because it holds after the last iteration). The fact that P is both a pre- and post-condition for the while loop is reflected in the conclusion of the rule.
There is one more rule, the rule of consequence, which allows to strengthen pre-conditions and weaken post-conditions:

$$\frac{\models (P \Rightarrow P') \qquad \{P'\}\ c\ \{Q'\} \qquad \models (Q' \Rightarrow Q)}{\{P\}\ c\ \{Q\}}\ \text{CONSEQUENCE}$$

These set of Hoare rules represent an inductive definition for a set of partial correctness statements $\{P\}\ c\ \{Q\}$. We will say that $\{P\}\ c\ \{Q\}$ is a *theorem in Hoare logic*, written $\vdash \{P\}\ c\ \{Q\}$, if we can build a finite proof tree for it.

3.5 Soundness and Completeness

At this point we have two kinds of partial correctness assertions:

- valid partial correctness statements $\models \{P\}\ c\ \{Q\}$, which hold for all stores and interpretations, according to the semantics of c, and
- Hoare logic theorems $\vdash \{P\}\ c\ \{Q\}$, that is, partial correctness statements that can be derived using the axioms and rules of Hoare logic.

The question is how do these sets relate to each other? More precisely, we have to answer two questions. First, is each Hoare logic theorem guaranteed to be valid partial correctness triple? In other words,

$$\text{does } \vdash \{P\}\ c\ \{Q\} \text{ imply } \models \{P\}\ c\ \{Q\}?$$

The answer is yes, and it shows that Hoare logic is *sound*. Soundness is important because it says that Hoare logic doesn't allow us to derive partial correctness assertions that actually don't hold. The proof of soundness requires induction on the derivations in $\vdash \{P\}\ c\ \{Q\}$ (we omit this proof).
The second question refers to the expressiveness and power of Hoare rules: can we always build a Hoare logic proof for each valid assertion? In other words,

$$\text{does } \models \{P\}\ c\ \{Q\} \text{ imply } \vdash \{P\}\ c\ \{Q\}?$$

The answer is a qualified yes: if $\models \{P\}\ c\ \{Q\}$ then there is a proof of $\vdash \{P\}\ c\ \{Q\}$ using the rules of Hoare logic, provided there are proofs for the validity of assertions that occur in the rule of consequence $\models (P \Rightarrow P')$ and $\models (Q' \Rightarrow Q)$. This result is known as the *relative completeness* of Hoare logic and is due to Cook (1974).

3.6 Example: Factorial

As an example illustrating how we can use Hoare logic to verify the correctness of a program, consider a program that computes the factorial of a number n:

$\{\mathtt{x} = n \wedge n > 0\}$
`y := 1;`
while `x > 0` **do** (
    `y := y * x;`
    `x := x - 1`
)
$\{\mathtt{y} = n!\}$

Because the derivation for this proof is somewhat large, we will go through the reasoning used to construct it step by step.

At the top level, the program is a sequence of an assignment and a loop. To use the SEQ rule, we need to find an assertion that holds after the assignment and before the loop. Examining the rule for while loops, we see that the assertion before the loop must be an invariant for the loop. Inspecting the loop we see that it builds the factorial up in $\mathtt{y}$ starting with n, then multiplying it by $n-1$, then $n-2$, etc. At each iteration, $\mathtt{x}$ contains the next value multiplied into $\mathtt{y}$, that is:

$$\mathtt{y} = n * (n-1) * \ldots * (\mathtt{x}+1)$$

If we multiply both sides of this equality by $\mathtt{x}!$ and re-write the equality we get $\mathtt{x}! * \mathtt{y} = n!$, which is an invariant for the loop. However, to make the proof go through, we need a slightly stronger invariant:

$$I = \mathtt{x}! * \mathtt{y} = n! \wedge \mathtt{x} \geqslant 0$$

Having identified a suitable loop invariant, let us take a step back and review where we are. We want to prove that our overall partial correctness specification is valid. To do this, we need to show two facts:

$$\{\mathtt{x} = n \wedge n > 0\}\ \mathtt{y} := \mathtt{1}\ \{I\} \tag{3.1}$$

$$\{I\}\ \textbf{while}\ \mathtt{x} > \mathtt{0}\ \textbf{do}\ (\mathtt{y} := \mathtt{y} * \mathtt{x};\ \mathtt{x} := \mathtt{x} - \mathtt{1})\ \{\mathtt{y} = n!\} \tag{3.2}$$

After showing that both (1) and (2) hold, we can use the rule SEQ to obtain the desired result.
To show (1), we use the ASSIGN axiom and obtain the following: $\{I[1/\mathtt{y}]\}\ \mathtt{y} := \mathtt{1}\ \{I\}$. Expanding this out, we obtain:

$$\{\mathtt{x}! * 1 = n! \wedge \mathtt{x} \geqslant 0\}\ \mathtt{y} := \mathtt{1}\ \{\mathtt{x}! * \mathtt{y} = n! \wedge \mathtt{x} \geqslant 0\}$$

With the following implication,

$$\mathtt{x} = n \wedge n > 0 \implies \mathtt{x}! * 1 = n! \wedge \mathtt{x} \geqslant 0,$$

(which can be shown by an easy calculation) we obtain (1) using the rule CONSEQUENCE.
Now let us prove (2). To use the WHILE rule, we need to show that I is an invariant for the loop:

$$\{I \wedge \mathtt{x} > 0\}\ \mathtt{y} := \mathtt{y} * \mathtt{x};\ \mathtt{x} := \mathtt{x} - \mathtt{1}\ \{I\} \tag{3.3}$$

We will show this by going backwards through the sequence of assignments:

$$\{(\mathtt{x}-1)! * \mathtt{y} = n! \wedge (\mathtt{x}-1) \geqslant 0\}\ \mathtt{x} := \mathtt{x} - \mathtt{1}\ \{I\} \tag{3.4}$$

$$\{(\mathtt{x}-1)! * \mathtt{y} * \mathtt{x} = n! \wedge (\mathtt{x}-1) \geqslant 0\}\ \mathtt{y} := \mathtt{y} * \mathtt{x}\ \{(\mathtt{x}-1)! * \mathtt{y} = n! \wedge (\mathtt{x}-1) \geqslant 0\} \tag{3.5}$$

Then, using the following implication:

$$I \wedge \mathtt{x} > 0 \implies (\mathtt{x}-1)! * \mathtt{y} * \mathtt{x} = n! \wedge (\mathtt{x}-1) \geqslant 0$$

we obtain (3) using CONSEQUENCE, (4), and (5). Thus, I is an invariant for the loop and so by WHILE we obtain,

$$\{I\}\ \textbf{while}\ \mathtt{x} > \mathtt{0}\ \textbf{do}\ (\mathtt{y} := \mathtt{y} * \mathtt{x};\ \mathtt{x} := \mathtt{x} - \mathtt{1})\ \{I \wedge \mathtt{x} \leqslant 0\}$$

To finish the proof, we just have to show

$$I \wedge \mathtt{x} \leqslant 0 \implies \mathtt{y} = n!$$

i.e.,

$$\mathtt{x}! * \mathtt{y} = n! \wedge \mathtt{x} \geqslant 0 \wedge \mathtt{x} \leqslant 0 \implies \mathtt{y} = n!$$

which holds as $\mathtt{x} \geqslant 0$ and $\mathtt{x} \leqslant 0$ implies $\mathtt{x} = 0$ and so $\mathtt{x}! = 1$. The result follows by CONSEQUENCE.
Chap ? in winscal has more info about this work plz

3.7 Decorated Programs

Doing a complete proof in Hoare Logic can feel overly verbose. The "core" of a proof consists of the preconditions and postconditions surrounding every command; with those, it's usually possible to infer the complete shape of the proof tree.

Decorating programs is a way to informally write down a Hoare logic proof of correctness. The idea is to insert assertion decorations between every "line" of the program. Using this informal evidence, you can reconstruct the full formal proof.

3.7.1 Informal Rules

We'll set out a few rules to check whether a decorated program represents a valid proof using *local consistency* checks. The rules, as you'll see, are informal reflections of the formal inference rules for Hoare logic.

Skip. For **skip**, the precondition and postcondition should be the same, like this:

$$\begin{array}{l}\{P\}\\ \textbf{skip}\\ \{P\}\end{array}$$

Sequence. For sequences, a new assertion R appears between the two commands. The two halves $\{P\}\ c_1\ \{R\}$ and $\{R\}\ c_2\ \{Q\}$ must be (recursively) locally consistent:

$$\begin{array}{l}\{P\}\\ c_1;\\ \{R\}\\ c_2\\ \{Q\}\end{array}$$

Assignment. Assignments are locally consistent when the precondition is the same as the postcondition except that it substitutes the assigned expression in for the variable:

$$\begin{array}{l}\{P[a/x]\}\\ x := a\\ \{P\}\end{array}$$

Conditions. An **if** is locally consistent when both branches are locally consistent after adding the branch condition to each:

$$\begin{array}{l}\{P\}\\ \textbf{if } b \textbf{ then}\\ \quad \{P \wedge b\}\\ \quad c_1\\ \quad \{Q\}\\ \textbf{else}\\ \quad \{P \wedge \neg b\}\\ \quad c_2\\ \quad \{Q\}\\ \{Q\}\end{array}$$

Loops. A **while** command should be decorated with a loop invariant:

$$\begin{array}{l}\{P\}\\ \textbf{while } b \textbf{ do}\\ \quad \{P \wedge b\}\\ \quad c\\ \quad \{P\}\\ \{P \wedge \neg b\}\end{array}$$

Implication. To capture the CONSEQUENCE rule, you can always write a (valid) implication to connect two commands:

$$\begin{array}{l}\{P\} \Rightarrow\\ \{Q\}\end{array}$$

Note:

It's possible to give a less redundant definition of these rules (less decorated), but this won't have the property of having a pre and post condition for every command.

3.7.2 Decorating a Program

These informal rules tell you how to verify whether a decorated program is correct, but they don't by themselves tell you how to construct those decorations (and thereby a proof). You can do this almost mechanically—except for loop invariants, which still require some creativity to concoct.
Here's an example program:

$$\textbf{while } (0 < \mathtt{y}) \textbf{ do } (\ \mathtt{x} := \mathtt{x} + 1;\ \mathtt{y} := \mathtt{y} - 1\ )$$

The first step is to decide what we want to prove about this program. So we write a precondition and postcondition for the whole program. Intuitively, the program adds y onto the initial value of the variable x, so we'll assert that:

$$\begin{array}{l}\{\mathtt{x} = m \wedge \mathtt{y} = n \wedge 0 \leqslant n\}\\ \textbf{while } (0 < \mathtt{y}) \textbf{ do } (\\ \quad \mathtt{x} := \mathtt{x} + 1;\\ \quad \mathtt{y} := \mathtt{y} - 1\\ )\\ \{\mathtt{x} = m + n\}\end{array}$$

This program is a while command, so the next step is to come up with a loop invariant. We'll set up the structure first. We need an assertion I where we can "connect" the loop's precondition and postcondition to our overall precondition and postcondition using implications, like this:

$$\begin{array}{l}\{\mathtt{x} = m \wedge \mathtt{y} = n \wedge 0 \leqslant n\} \Rightarrow\\ \{I\}\\ \textbf{while } (0 < \mathtt{y}) \textbf{ do } (\\ \quad \{I \wedge 0 < y\}\\ \quad \mathtt{x} := \mathtt{x} + 1;\\ \quad \mathtt{y} := \mathtt{y} - 1\\ \quad \{I\}\\ )\\ \{I \wedge 0 \not< \mathtt{y}\} \Rightarrow\\ \{\mathtt{x} = m + n\}\end{array}$$

On every iteration of the loop, the variable x is the sum we want, $m + n$, less the current value of y, which is the number of iterations remaining. So we'll define the invariant I like this:

$$I ::= (\mathtt{x} = m + n - \mathtt{y}) \wedge 0 \leqslant \mathtt{y}$$

The top implication follows because $n - \mathtt{y} = 0$, and the bottom implication is valid because $0 \not< \mathtt{y}$ and $0 \leqslant \mathtt{y}$ together imply $\mathtt{y} = 0$.
To finish decorating the program, we need an assertion between the two lines in the body of the loop. By the rule for assignments, we know that this must be $I[\mathtt{y} - 1/\mathtt{y}]$, or:

$$P_1 ::= (\mathtt{x} = m + n - (\mathtt{y} - 1)) \wedge (0 \leqslant \mathtt{y} - 1)$$

To make the assignment to x locally consistent, then, its precondition must be $P_1[\mathtt{x} + 1/\mathtt{x}]$, or:

$$P_2 ::= (\mathtt{x} + 1 = m + n - (\mathtt{y} - 1)) \wedge (0 \leqslant \mathtt{y} - 1)$$

It's straightforward to see that the precondition at the top of the loop, $I \wedge 0 < \mathtt{y}$, implies this new assertion P_2. Now we can write our complete decorated program using these definitions:

$\{\mathrm{x} = m \wedge \mathrm{y} = n \wedge 0 \leqslant n\} \Rightarrow$
$\{I\}$
while $(0 < \mathrm{y})$ **do** (
    $\{I \wedge 0 < y\} \Rightarrow$
    $\{P_2\}$
    $\mathrm{x} := \mathrm{x} + 1;$
    $\{P_1\}$
    $\mathrm{y} := \mathrm{y} - 1$
    $\{I\}$
)
$\{I \wedge 0 \not< \mathrm{y}\} \Rightarrow$
$\{\mathrm{x} = m + n\}$

3.8 Weakest Preconditions

Previously, we discussed the issue of completeness—i.e., whether it is possible to derive every valid partial correctness specification using the axioms and rules of Hoare logic. Unfortunately, if treated as a pure deductive system, Hoare logic cannot be complete. To see why, consider the following partial correctness specifications:

$$\{\textbf{true}\}\ \textbf{skip}\ \{P\} \qquad \{\textbf{true}\}\ c\ \{\textbf{false}\}$$

The first is valid if and only if the assertion P is valid while the second is valid if and only if the command c does not halt.
It turns out that the culprit is the CONSEQUENCE rule,

$$\frac{\models (P \Rightarrow P') \qquad \{P'\}\ c\ \{Q'\} \qquad \models (Q' \Rightarrow Q)}{\{P\}\ c\ \{Q\}}\ \text{CONSEQUENCE}$$

which includes two premises about the validity of implications between the assertions involved. Proving those implications for our two examples would require proving arbitrary logical predicates P and proving that arbitrary IMP programs c terminate. By Gödel's incompleteness theorem, we know that there is no consistent mathematical proof system that can do the former. Therefore, Hoare logic cannot be complete in that sense.
On the other hand, Hoare logic does enjoy this property:

Theorem 3.8.1 Relative Completeness of Hoare Logic
$\forall P, Q \in \textbf{Assn}, c \in \textbf{Com}.\ \models \{P\}\ c\ \{Q\}$ implies $\vdash \{P\}\ c\ \{Q\}$.

This result states that Hoare logic is no more incomplete than the language of assertions—i.e., if we had an oracle that could decide the validity of assertions, then we could decide the validity of partial correctness specifications. In other words, for any valid Hoare triple, there exists a proof in Hoare logic—as long as you have some external way to decide the validity of the premises to the CONSEQUENCE rule.

Cook's proof of relative completeness depends on the notion of *weakest liberal preconditions*. Given a command c and a postcondition Q, the weakest liberal precondition is the weakest assertion P such that $\{P\}\ c\ \{Q\}$ is valid. Here, "weakest" means that any other valid precondition implies P. That is, P most accurately describes input states for which c either does not terminate or ends up in a state satisfying Q.

Definition 3.8.1: Weakest Liberal Precondition

Formally, an assertion P is a weakest liberal precondition of c and Q if:

$$\forall \sigma, I.\ \sigma \models_I P \iff (C[\![c]\!]\,\sigma) \text{ is undefined } \vee\ (C[\![c]\!]\sigma) \models_I Q$$

Note:
The interpretation is the same to check P and Q, meaning the program can't change the set of logical variables.

We write $\mathrm{wlp}(c, Q)$ for the weakest liberal precondition of command c and postcondition Q. From left-to-right, the formula above states that $\mathrm{wlp}(c, Q)$ is a valid precondition: $\models \{P\}\ c\ \{Q\}$. The right-to-left implication says it

is the weakest valid precondition: if another assertion R satisfies $\models \{R\}\ c\ \{Q\}$, then R implies P. It can be shown that weakest liberal preconditions are unique modulo equivalence.
We can calculate the weakest liberal precondition of a command as follows:

$$\begin{aligned}
\mathrm{wlp}(\textbf{skip}, P) &= P \\
\mathrm{wlp}(x := a, P) &= P[a/x] \\
\mathrm{wlp}((c_1; c_2), P) &= \mathrm{wlp}(c_1, \mathrm{wlp}(c_2, P)) \\
\mathrm{wlp}(\textbf{if } b \textbf{ then } c_1 \textbf{ else } c_2, P) &= (b \implies \mathrm{wlp}(c_1, P)) \wedge (\neg b \implies \mathrm{wlp}(c_2, P))
\end{aligned}$$

The definition of $\mathrm{wlp}(\textbf{while } b \textbf{ do } c, P)$ is slightly more complicated—it encodes the weakest liberal precondition for each iteration of the loop. To give the intuition, first define the weakest liberal precondition for a loop that terminates in i steps as follows:

$$\begin{aligned}
F_0(P) &= \textbf{true} \\
F_{i+1}(P) &= (\neg b \implies P) \wedge (b \implies \mathrm{wlp}(c, F_i(P)))
\end{aligned}$$

We can then express the weakest liberal precondition using an infinitary conjunction:

$$\mathrm{wlp}(\textbf{while } b \textbf{ do } c, P) = \bigwedge_i F_i(P)$$

See Winskel Chapter 7 for the details of how to encode the weakest liberal precondition for a while loop as an ordinary assertion.
To check that our definition is correct, we can prove (how?) that it yields a valid partial correctness specification:

Lenma 3.8.1 Correctness of Weakest Preconditions

$$\forall c \in \textbf{Com}, Q \in \textbf{Assn}.\ \models \{\mathrm{wlp}(c, Q)\}\ c\ \{Q\} \text{ and } \forall R \in \textbf{Assn}.\ \models \{R\}\ c\ \{Q\} \text{ implies } (R \implies \mathrm{wlp}(c, Q))$$

It is not hard to prove that it also yields a provable specification:

Lenma 3.8.2 Provability of Weakest Preconditions
$\forall c \in \textbf{Com}, Q \in \textbf{Assn}.\ \vdash \{\mathrm{wlp}(c, Q)\}\ c\ \{Q\}$

Relative completeness follows by a simple argument:

Sketch: Let c be a command and let P and Q be assertions such that the partial correctness specification $\{P\}\ c\ \{Q\}$ is valid. By Lemma 1 we have $\models P \implies \mathrm{wlp}(c, Q)$. By Lemma 2 we have $\vdash \{\mathrm{wlp}(c, Q)\}\ c\ \{Q\}$. We conclude $\vdash \{P\}\ c\ \{Q\}$ using the CONSEQUENCE rule.

Chapter 4

Separation Logic

So far, we have looked at a version of Hoare logic that supports reasoning about IMP programs in terms of pre- and post-conditions on stores. In this lecture, we consider a language with pointers and dynamic memory allocation. We define a big-step operational semantics for the language as a relation on program states (σ, h) with a store and a heap. We show how Hoare logic fails to support modular reasoning due to aliasing between pointers. Finally, we introduce *separation logic*, which extends Hoare logic's assertions with predicates on heaps and provides a *frame* rule that supports modular proofs.

4.1 IMP with Heaps

As a first step, we extend IMP with standard commands for manipulating values on the heap. Note: in this lecture we will write e for arithmetic expressions rather than a, as it is more intuitive:

$$\begin{aligned} c \ ::= \ & \textbf{skip} \mid x := e \mid c_1; c_2 \mid \textbf{if } b \textbf{ then } c_1 \textbf{ else } c_2 \mid \textbf{while } b \textbf{ do } c \\ & \mid x := \textbf{new}(e) \mid \textbf{free}(e) \mid x := *e \mid *e_1 := e_2 \end{aligned}$$

The new commands in this language include:

- $x := \textbf{new}(e)$, which evaluates e to a value v, allocates storage for v on the heap, and assigns the pointer to that storage to x in the store;
- $x := *e$, which evaluates e to a pointer p, loads the value v associated with p from the heap, and assigns v to x in the store;
- $*e_1 := e_2$, which evaluates e_1 to a pointer p and e_2 to a value v, and then stores v at the location associated with p on the heap; and
- $\textbf{free}(e)$, which evaluates e to a pointer p and deallocates the heap storage associated with p.

4.2 Big-Step Operational Semantics

To define the semantics of this extension of IMP, we will need states that keep track both of the local variables as well as the heap. We will model states as pairs (σ, h) where

- $\sigma \in \text{Var} \to \mathbb{Z}$ is the store, which for simplicity we take to be a total function, and
- $h \in \text{Addr} \rightharpoonup_{\text{fin}} \mathbb{Z}$ is the heap, a finite partial function.

Here $\text{Addr} \subseteq \mathbb{Z} \setminus \{0\}$ is the set of addresses. Pointers are just integers, so they can be stored in variables, stored on the heap, and computed with arithmetic expressions. We exclude 0 so that it can serve as the null pointer: no cell is ever stored at address 0. As usual, we write $\text{dom}(h)$ for the domain of the heap and $h[p \mapsto v]$ for the heap that maps p to v and otherwise behaves like h. In the formal semantics, commands evaluate in one big step to an outcome ω, which is either a final state or a memory error:

$$\langle \sigma, h, c \rangle \Downarrow \omega \qquad\qquad \omega \ ::= \ (\sigma', h') \mid \text{err}$$

while arithmetic and boolean expressions remain pure,

$$\langle \sigma, e \rangle \Downarrow v$$

Basic commands

$$\frac{}{\langle \sigma, h, \mathbf{skip} \rangle \Downarrow (\sigma, h)}\ \textsc{Skip} \qquad \frac{\langle \sigma, e \rangle \Downarrow v}{\langle \sigma, h, x := e \rangle \Downarrow (\sigma[x \mapsto v], h)}\ \textsc{Assign}$$

$$\frac{\langle \sigma, h, c_1 \rangle \Downarrow (\sigma_1, h_1) \qquad \langle \sigma_1, h_1, c_2 \rangle \Downarrow \omega}{\langle \sigma, h, c_1; c_2 \rangle \Downarrow \omega}\ \textsc{Seq}$$

Conditionals and loops

$$\frac{\langle \sigma, b \rangle \Downarrow \text{true} \qquad \langle \sigma, h, c_1 \rangle \Downarrow \omega}{\langle \sigma, h, \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2 \rangle \Downarrow \omega}\ \textsc{If-True} \qquad \frac{\langle \sigma, b \rangle \Downarrow \text{false} \qquad \langle \sigma, h, c_2 \rangle \Downarrow \omega}{\langle \sigma, h, \mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2 \rangle \Downarrow \omega}\ \textsc{If-False}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \text{false}}{\langle \sigma, h, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow (\sigma, h)}\ \textsc{While-False}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \text{true} \qquad \langle \sigma, h, c \rangle \Downarrow (\sigma_1, h_1) \qquad \langle \sigma_1, h_1, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \omega}{\langle \sigma, h, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \omega}\ \textsc{While-True}$$

Heap commands

$$\frac{\langle \sigma, e \rangle \Downarrow p \qquad p \in \mathrm{dom}(h) \qquad h(p) = v}{\langle \sigma, h, x := *e \rangle \Downarrow (\sigma[x \mapsto v], h)}\ \textsc{Load} \qquad \frac{\langle \sigma, e_1 \rangle \Downarrow p \qquad \langle \sigma, e_2 \rangle \Downarrow v \qquad p \in \mathrm{dom}(h)}{\langle \sigma, h, *e_1 := e_2 \rangle \Downarrow (\sigma, h[p \mapsto v])}\ \textsc{Store}$$

$$\frac{\langle \sigma, e \rangle \Downarrow v \qquad p \in \mathrm{Addr} \setminus \mathrm{dom}(h)}{\langle \sigma, h, x := \mathbf{new}(e) \rangle \Downarrow (\sigma[x \mapsto p], h[p \mapsto v])}\ \textsc{New} \qquad \frac{\langle \sigma, e \rangle \Downarrow p \qquad p \in \mathrm{dom}(h)}{\langle \sigma, h, \mathbf{free}(e) \rangle \Downarrow (\sigma, h|_{\mathrm{dom}(h) \setminus \{p\}})}\ \textsc{Free}$$

Here $h|_A$ is the restriction of h to the addresses in A. Note that New may choose any unused address p, so the semantics is nondeterministic.

Memory errors A load, store, or free at an address that is not allocated produces a memory error, and errors propagate through sequencing and loops:

$$\frac{\langle \sigma, e \rangle \Downarrow p \qquad p \notin \mathrm{dom}(h)}{\langle \sigma, h, x := *e \rangle \Downarrow \mathrm{err}}\ \textsc{Load-Err} \qquad \frac{\langle \sigma, e_1 \rangle \Downarrow p \qquad p \notin \mathrm{dom}(h)}{\langle \sigma, h, *e_1 := e_2 \rangle \Downarrow \mathrm{err}}\ \textsc{Store-Err}$$

$$\frac{\langle \sigma, e \rangle \Downarrow p \qquad p \notin \mathrm{dom}(h)}{\langle \sigma, h, \mathbf{free}(e) \rangle \Downarrow \mathrm{err}}\ \textsc{Free-Err} \qquad \frac{\langle \sigma, h, c_1 \rangle \Downarrow \mathrm{err}}{\langle \sigma, h, c_1; c_2 \rangle \Downarrow \mathrm{err}}\ \textsc{Seq-Err}$$

$$\frac{\langle \sigma, b \rangle \Downarrow \text{true} \qquad \langle \sigma, h, c \rangle \Downarrow \mathrm{err}}{\langle \sigma, h, \mathbf{while}\ b\ \mathbf{do}\ c \rangle \Downarrow \mathrm{err}}\ \textsc{While-Err}$$

The rules for Seq, If, and While-True above already pass along an error from their last premise. Since $0 \notin \mathrm{Addr}$, any load, store, or free at address 0 produces err. No other command can fail: the store is total, so expressions always evaluate, and we assume that Addr is infinite, so New can always find an unused address in the finite heap. Thanks to the outcome err, a program that crashes is now distinguishable from one that runs forever, which has no derivation at all.

4.3 The Rule of Constancy

The following rule is admissible in standard Hoare logic:

$$\frac{\vdash \{P\}\ c\ \{Q\} \qquad \mathrm{fv}(R) \cap \mathrm{mod}(c) = \emptyset}{\vdash \{P \wedge R\}\ c\ \{Q \wedge R\}}\ \textsc{Constancy}$$

Here $\mathrm{fv}(R)$ are the free variables of R and $\mathrm{mod}(c)$ are the variables that c may modify. For example, $\mathrm{fv}(y = 0) = \{y\}$ and $\mathrm{mod}(x := x + 1) = \{x\}$. Intuitively, the rule of constancy captures a form of local reasoning. It allows us to first prove a simple Hoare triple and then extend it to a more complicated triple by conjoining the same predicate to the pre- and post-conditions. For instance, using this rule we can strengthen

$$\{x > 0\}\ x := x + 1\ \{x > 1\}$$

to

$$\{x > 0 \wedge y = 0\}\ x := x + 1\ \{x > 1 \wedge y = 0\}$$

Free and Modified Variables The function $\mathrm{fv}(P)$ returns the set of program variables that occur free in assertion P.

$$\begin{aligned}
\mathrm{fv}(n) = \mathrm{fv}(\mathrm{true}) = \mathrm{fv}(\mathrm{false}) &= \emptyset \\
\mathrm{fv}(x) &= \{x\} \\
\mathrm{fv}(a_1 + a_2) = \mathrm{fv}(a_1 \times a_2) = \mathrm{fv}(a_1 \leq a_2) &= \mathrm{fv}(a_1) \cup \mathrm{fv}(a_2) \\
\mathrm{fv}(P \wedge Q) = \mathrm{fv}(P \vee Q) = \mathrm{fv}(P \Rightarrow Q) &= \mathrm{fv}(P) \cup \mathrm{fv}(Q) \\
\mathrm{fv}(\neg P) = \mathrm{fv}(\forall i.\, P) = \mathrm{fv}(\exists i.\, P) &= \mathrm{fv}(P)
\end{aligned}$$

The quantifier cases do not remove anything, because i is a logical variable, not a program variable. The function $\mathrm{mod}(c)$ returns the set of program variables modified by command c.

$$\begin{aligned}
\mathrm{mod}(\mathbf{skip}) = \mathrm{mod}(*e_1 := e_2) = \mathrm{mod}(\mathbf{free}(e)) &= \emptyset \\
\mathrm{mod}(x := e) = \mathrm{mod}(x := *e) = \mathrm{mod}(x := \mathbf{new}(e)) &= \{x\} \\
\mathrm{mod}(c_1; c_2) = \mathrm{mod}(\mathbf{if}\ b\ \mathbf{then}\ c_1\ \mathbf{else}\ c_2) &= \mathrm{mod}(c_1) \cup \mathrm{mod}(c_2) \\
\mathrm{mod}(\mathbf{while}\ b\ \mathbf{do}\ c) &= \mathrm{mod}(c)
\end{aligned}$$

Issues Related to Aliasing Unfortunately, the rule of constancy is not sound when we extend our language of assertions with heap predicates. For example, suppose that we add an assertion $e_1 \hookrightarrow e_2$, which says that the heap contains a cell at address e_1 holding e_2 (and possibly other cells as well). Now consider trying to use the rule of constancy to add a "constant fact" about the heap. We might first prove the following triple:

$$\{\, x \hookrightarrow - \,\}\ *x := 4\ \{\, x \hookrightarrow 4 \,\}.$$

and then use the rule of constancy to obtain:

$$\{\, x \hookrightarrow - \ \wedge\ y \hookrightarrow 3 \,\}\ *x := 4\ \{\, x \hookrightarrow 4 \ \wedge\ y \hookrightarrow 3 \,\}.$$

Here $\mathrm{mod}(*x := 4) = \emptyset$, so the side condition of the rule is satisfied trivially. However, if x and y are aliases for each other, then the postcondition is false. Of course, it is possible to complicate the pre- and post-condition to capture disjointness predicates, but this quickly becomes impractical—in principle, every pointer on the heap might be aliased to every other pointer, which leads to a combinatorial explosion. Such complications motivated the development of separation logic.

4.4 Heap Assertions

Before defining separation logic, let us extend our language of assertions with heap predicates:

$$H, J, K \ ::=\ \mathrm{emp} \mid [P] \mid e_1 \hookrightarrow e_2 \mid H_1 \star H_2 \mid H_1 \mathrel{-\!\!*} H_2 \mid H_1 \mathrel{-\!\!\!\wedge} H_2 \mid H_1 \mathrel{-\!\!\!\vee} H_2 \mid \forall i.\, H \mid \exists i.\, H$$

Intuitively, these predicates can be understood as follows.

- emp : heap is empty,
- $[P]$: heap is empty and P holds,
- $e_1 \hookrightarrow e_2$: heap consists of exactly one cell with pointer e_1 and value e_2 (this is stricter than the informal reading of $\hookrightarrow$ in the previous section),

- $H_1 \star H_2$: heap splits into two disjoint pieces, one satisfying H_1 and the other H_2,
- $H_1 \mathbin{\curlywedge} H_2$ and $H_1 \mathbin{\curlyvee} H_2$ are heap assertion versions of the standard boolean connectives, and
- $\forall i.\, H$ and $\exists i.\, H$ are heap assertion versions of the standard quantifiers.

More formally, we can model their semantics as follows:

$$
\begin{array}{ll}
(\sigma, h) \models_I \text{emp} & \text{if } h = \emptyset \\
(\sigma, h) \models_I [P] & \text{if } h = \emptyset \wedge \sigma \models_I P \\
(\sigma, h) \models_I e_1 \hookrightarrow e_2 & \text{if } h = \{(p, v)\} \text{ where } \langle \sigma, e_1 \rangle_I \Downarrow p \text{ and } \langle \sigma, e_2 \rangle_I \Downarrow v \\
(\sigma, h) \models_I H_1 \star H_2 & \text{if } \exists h_1, h_2.\ h = h_1 \uplus h_2 \wedge (\sigma, h_1) \models_I H_1 \wedge (\sigma, h_2) \models_I H_2 \\
(\sigma, h) \models_I H_1 \mathbin{\curlywedge} H_2 & \text{if } (\sigma, h) \models_I H_1 \text{ and } (\sigma, h) \models_I H_2 \\
(\sigma, h) \models_I H_1 \mathbin{\curlyvee} H_2 & \text{if } (\sigma, h) \models_I H_1 \text{ or } (\sigma, h) \models_I H_2 \\
(\sigma, h) \models_I \forall i.\, H & \text{if } (\sigma, h) \models_{I[i \mapsto v]} H \text{ for all } v \\
(\sigma, h) \models_I \exists i.\, H & \text{if } (\sigma, h) \models_{I[i \mapsto v]} H \text{ for some } v
\end{array}
$$

Here $h_1 \uplus h_2$ is the union of heaps h_1 and h_2 with disjoint domains (it is undefined if they overlap). Free variables extend to heap assertions in the obvious way, e.g., $\mathrm{fv}(e_1 \hookrightarrow e_2) = \mathrm{fv}(e_1) \cup \mathrm{fv}(e_2)$, $\mathrm{fv}(\text{emp}) = \emptyset$, and $\mathrm{fv}([P]) = \mathrm{fv}(P)$. Quantifiers bind logical variables (never program variables), which range over integers, including addresses, and over finite lists of integers; I maps each to a value of the right kind. We typically write i, j, a, b for integers and L for lists. We will often abbreviate $e \hookrightarrow - \triangleq \exists i.\, e \hookrightarrow i$. As heaps only contain addresses, $0 \hookrightarrow v$ is unsatisfiable for every v: there is no heap whose single cell is at address 0. Similarly, predicates like $x \hookrightarrow 5 \star x \hookrightarrow 5$ are unsatisfiable as the separating conjunction operator, $\star$, enforces disjointness—even when the two cells would hold the same value.

Equivalence and entailment. Two heap assertions are equivalent, written $H_1 \equiv H_2$, if they hold in exactly the same states:

$$H_1 \equiv H_2 \triangleq \forall \sigma, h, I.\ (\sigma, h) \models_I H_1 \iff (\sigma, h) \models_I H_2.$$

(Note that it is not enough for H_1 and H_2 to be both valid or both invalid—for example, $x \hookrightarrow 1$ and $x \hookrightarrow 2$ are both invalid, but they are certainly not equivalent.) Similarly, H_1 *entails* H_2, written $H_1 \Rrightarrow H_2$, if every state satisfying H_1 also satisfies H_2. Separating conjunction satisfies several familiar algebraic laws:

$$
\begin{array}{rl}
H \star J \equiv J \star H & \text{(commutativity)} \\
(H \star J) \star K \equiv H \star (J \star K) & \text{(associativity)} \\
H \star \text{emp} \equiv H & \text{(unit)}
\end{array}
$$

However, $\star$ and $\mathbin{\curlywedge}$ behave differently. In general, $[P] \star H \not\equiv [P] \mathbin{\curlywedge} H$: the left-hand side says that P holds and H describes the whole heap, while the right-hand side requires the heap to be empty. For instance, $[\text{true}] \star x \hookrightarrow 1$ is satisfiable, but $[\text{true}] \mathbin{\curlywedge} x \hookrightarrow 1$ is not. When H is emp, the two do coincide: $[P] \star \text{emp} \equiv [P] \mathbin{\curlywedge} \text{emp}$. In practice, $[P] \star H$ is the idiomatic way to state a pure fact P alongside a heap assertion H, and we will use entailments such as $[P] \star H \Rrightarrow H$ freely.

4.5 Valid Triples

Now we define the notion of valid triples. We use the standard definition for separation logic, which is partial correctness with *fault avoidance*, following Reynolds and O'Hearn: a valid triple guarantees that the program never produces a memory error, and that if it terminates, the postcondition holds. The explicit outcome err is what makes this possible. If memory errors simply got stuck, a crashing program would have no derivation, just like a program that runs forever, and a partial correctness triple would hold vacuously for it. With fault avoidance, {emp} **free**(x) {false} is not valid, because **free**(x) produces err on the empty heap. On the other hand, {emp} **while** true **do skip** {false} is valid: the loop never crashes and never terminates. As usual for partial correctness, termination must be proved separately if it is needed. **Definition (Valid Triple).** A triple is *valid*, written $\models \{H\}\ c\ \{J\}$, if for all σ, h, and I with $\sigma, h \models_I H$:

- $\langle \sigma, h, c \rangle \Downarrow \text{err}$ is not derivable, and
- every σ', h' with $\langle \sigma, h, c \rangle \Downarrow (\sigma', h')$ satisfies $\sigma', h' \models_I J$.

Because New makes the semantics nondeterministic, both conditions quantify over all executions: a valid triple rules out an error for every possible choice of fresh addresses. Note that the precondition H describes the entire heap in which c runs, and c must not produce an error in any such heap. So a valid triple implicitly says that c only accesses cells described by H—its *footprint*. For example, $\{x \hookrightarrow 3\} \ *x := 4 \ \{x \hookrightarrow 4\}$ is valid, but $\{\text{emp}\} \ *x := 4 \ \{\text{emp}\}$ is not, because $*x := 4$ produces err on the empty heap.

4.6 The Frame Rule

A major appeal of separation logic is that it supports the so-called frame rule:

$$\frac{\vdash \{H\} \ c \ \{J\} \qquad \text{fv}(K) \cap \text{mod}(c) = \emptyset}{\vdash \{H \star K\} \ c \ \{J \star K\}} \ \textsc{Frame}$$

The rule captures modular reasoning about the heap. Intuitively, we first prove a specification for c using only the "heap" footprint it needs. Then the command can be framed in any larger context obtained by combining the footprint with a disjoint heap.

Soundness. Why is the frame rule sound? Write $h \perp h_0$ when h and h_0 have disjoint domains. The proof rests on four facts, the first three about the semantics (each proved by induction on derivations) and the last about assertions (proved by induction on the assertion):

- *Safety monotonicity*: if $\langle \sigma, h, c \rangle \Downarrow \text{err}$ is not derivable, then neither is $\langle \sigma, h \uplus h_0, c \rangle \Downarrow \text{err}$, for any $h_0 \perp h$. (If a load, store, or free finds its address in h, it also finds it in $h \uplus h_0$.)
- *Frame property*: if $\langle \sigma, h, c \rangle \Downarrow \text{err}$ is not derivable and $\langle \sigma, h \uplus h_0, c \rangle \Downarrow (\sigma', h')$, then $h' = h'_1 \uplus h_0$ for some h'_1 with $\langle \sigma, h, c \rangle \Downarrow (\sigma', h'_1)$. In other words, a run in the larger heap leaves h_0 untouched and behaves like a run in h.
- *Unmodified variables*: if $\langle \sigma, h, c \rangle \Downarrow (\sigma', h')$, then $\sigma'(y) = \sigma(y)$ for every $y \notin \text{mod}(c)$.
- *Coincidence* (also called the agreement or relevance lemma): if σ and σ' agree on $\text{fv}(K)$, then $\sigma, h \models_I K$ if and only if $\sigma', h \models_I K$. That is, an assertion depends on the store only through its free variables.

The interesting case of the frame property is New. The run in $h \uplus h_0$ allocates some $p \notin \text{dom}(h \uplus h_0)$. Since $\text{dom}(h) \subseteq \text{dom}(h \uplus h_0)$, this p is also unused in h, so the run in h alone can pick the same p, and the results match: $(h \uplus h_0)[p \mapsto v] = h[p \mapsto v] \uplus h_0$. This is exactly why New is allowed to choose any unused address. With a deterministic allocator, say $p = 1 + \max \text{dom}(h)$, the two runs could allocate different addresses, and the frame rule would be unsound: on the empty heap the allocator returns 1, so $\{\text{emp}\} \ x := \textbf{new}(1) \ \{[x = 1] \star x \hookrightarrow 1\}$ would be valid, but framing in $1 \hookrightarrow 5$ gives a false triple, because the allocator would now return 2. **Theorem (Soundness of Frame).** If $\models \{H\} \ c \ \{J\}$ and $\text{fv}(K) \cap \text{mod}(c) = \emptyset$, then $\models \{H \star K\} \ c \ \{J \star K\}$. *Proof.* Suppose $\sigma, h \models_I H \star K$. Then $h = h_1 \uplus h_0$ with $\sigma, h_1 \models_I H$ and $\sigma, h_0 \models_I K$. By the premise, $\langle \sigma, h_1, c \rangle \Downarrow \text{err}$ is not derivable, so by safety monotonicity neither is $\langle \sigma, h, c \rangle \Downarrow \text{err}$. Now suppose $\langle \sigma, h, c \rangle \Downarrow (\sigma', h')$. By the frame property, $h' = h'_1 \uplus h_0$ with $\langle \sigma, h_1, c \rangle \Downarrow (\sigma', h'_1)$, so $\sigma', h'_1 \models_I J$ by the premise. By the unmodified-variables fact, σ' agrees with σ on $\text{fv}(K)$, since $\text{fv}(K) \cap \text{mod}(c) = \emptyset$; so by coincidence we still have $\sigma', h_0 \models_I K$. Hence $\sigma', h' \models_I J \star K$. ∎

Why fault avoidance? The proof uses the absence of errors in an essential way. Suppose instead that memory errors simply got stuck, and triples were ordinary partial correctness statements. Then $\{\text{emp}\} \ \textbf{free}(x) \ \{\text{emp}\}$ would be valid vacuously, since $\textbf{free}(x)$ has no result on the empty heap. But framing in $x \hookrightarrow 3$ would give $\{x \hookrightarrow 3\} \ \textbf{free}(x) \ \{x \hookrightarrow 3\}$, which is false: the free succeeds and the cell is gone. Fault avoidance rules out the first triple, because it forces a command to own every cell it touches.

Baking in the frame rule. Some modern presentations, such as Charguéraud's, take a different approach and build the frame rule into the definition of triples. There, $\{H\} \ c \ \{J\}$ is defined to be valid if, for every K with $\text{fv}(K) \cap \text{mod}(c) = \emptyset$, the triple $\{H \star K\} \ c \ \{J \star K\}$ is valid in the sense above. The frame rule is then sound by construction, at the cost of a less direct definition.

4.7 Small Axioms for Heap Commands

We can define "small" axioms for the heap-manipulating commands as follows.

$$\frac{x \notin \mathrm{fv}(e_1) \cup \mathrm{fv}(e_2)}{\{\, e_1 \hookrightarrow e_2 \,\}\ x := *e_1\ \{\, [x = e_2] \star e_1 \hookrightarrow e_2 \,\}}\ \textsc{Load} \qquad \frac{}{\{\, e_1 \hookrightarrow - \,\}\ *e_1 := e_2\ \{\, e_1 \hookrightarrow e_2 \,\}}\ \textsc{Store}$$

$$\frac{x \notin \mathrm{fv}(e)}{\{\, \mathrm{emp} \,\}\ x := \mathbf{new}(e)\ \{\, x \hookrightarrow e \,\}}\ \textsc{Alloc} \qquad \frac{}{\{\, e \hookrightarrow - \,\}\ \mathbf{free}(e)\ \{\, \mathrm{emp} \,\}}\ \textsc{Free}$$

In Alloc, the program variable x ends up holding the address of the new cell, so $x \hookrightarrow e$ describes that cell. The side condition is needed because e is interpreted in the final store, where x has been overwritten. A full treatment of separation logic has other proof rules similar to Hoare logic. In the examples below we also use the rules for sequencing and consequence, where consequence is stated using entailment ($\Rrightarrow$) between heap assertions, and the familiar invariant rule for loops:

$$\frac{\{[b] \star I\}\ c\ \{I\}}{\{I\}\ \mathbf{while}\ b\ \mathbf{do}\ c\ \{[\neg b] \star I\}}\ \textsc{While}$$

Note how the loop test is combined with the invariant using $\star$: $[b] \star I$ says that b holds and I describes the heap. Using $[b] \wedge I$ instead would be wrong: it would also require an empty heap.

4.8 Example: Swap

The following program swaps the contents of the cells at p and q:

$$t := *p;\ u := *q;\ *p := u;\ *q := t$$

A natural specification is the following, where a and b are logical variables:

$$\{\, p \hookrightarrow a \star q \hookrightarrow b \,\}\ \mathrm{swap}\ \{\, p \hookrightarrow b \star q \hookrightarrow a \,\}.$$

We can prove it using the small axioms, framing away the cells (and pure facts) each command does not touch. Each use of Frame is allowed, since the frame never mentions the assigned variable:

$\{\, p \hookrightarrow a \star q \hookrightarrow b \,\}$
$t := *p;$ Load, framing $q \hookrightarrow b$
$\{\, [t = a] \star p \hookrightarrow a \star q \hookrightarrow b \,\}$
$u := *q;$ Load, framing $[t = a] \star p \hookrightarrow a$
$\{\, [t = a] \star [u = b] \star p \hookrightarrow a \star q \hookrightarrow b \,\}$
$*p := u;$ Store, framing everything but $p \hookrightarrow a$
$\{\, [t = a] \star [u = b] \star p \hookrightarrow u \star q \hookrightarrow b \,\}$
$*q := t$ Store, framing everything but $q \hookrightarrow b$
$\{\, [t = a] \star [u = b] \star p \hookrightarrow u \star q \hookrightarrow t \,\} \Rrightarrow$
$\{\, p \hookrightarrow b \star q \hookrightarrow a \,\}$

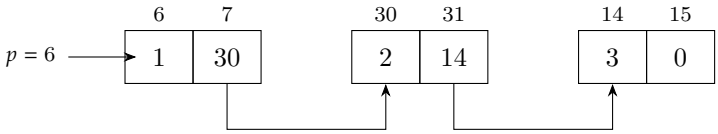
(Before each Store we also weaken $p \hookrightarrow a$ to $p \hookrightarrow -$, and similarly for q.)

Is this the only specification? No! Because $\star$ forces the two cells to be disjoint, the specification above says nothing about the case where p and q are aliases. In that case the program is the identity, which we can specify as follows:

$$\{\, [p = q] \star p \hookrightarrow a \,\}\ \mathrm{swap}\ \{\, [p = q] \star p \hookrightarrow a \,\}.$$

4.9 Linked Lists

Representation. We represent a linked list as a sequence of nodes. The node at address p occupies two consecutive cells: its value is stored at p and the address of the next node is stored at $p+1$. We use 0 as the null pointer; recall that $0 \notin \text{Addr}$, so 0 is never the address of a cell. For example, the following heap holds the list $[1,2,3]$ at address 6:



As a finite map, it is $h = \{6 \mapsto 1,\ 7 \mapsto 30,\ 30 \mapsto 2,\ 31 \mapsto 14,\ 14 \mapsto 3,\ 15 \mapsto 0\}$. Nodes need not be adjacent or in order; only the pointers matter. (Our **new**(e) allocates a single cell. A real language would allocate a whole node at once, but we will not need allocation below.)

A list predicate. To reason about lists, we define a heap predicate $\text{list}(p, L)$, meaning "p points to a list holding the mathematical list L," by recursion on L:

$$\begin{aligned} \text{list}(p, [\,]) &\triangleq [p = 0] \\ \text{list}(p, v :: L) &\triangleq \exists q.\ p \hookrightarrow v \star (p+1) \hookrightarrow q \star \text{list}(q, L) \end{aligned}$$

Unfolding the definition step by step:

$$\begin{aligned} & \text{list}(6, [1,2,3]) \\ \equiv\ & \exists q_1.\ 6 \hookrightarrow 1 \star 7 \hookrightarrow q_1 \star \text{list}(q_1, [2,3]) \\ \equiv\ & \exists q_1, q_2.\ 6 \hookrightarrow 1 \star 7 \hookrightarrow q_1 \star q_1 \hookrightarrow 2 \star (q_1+1) \hookrightarrow q_2 \star \text{list}(q_2, [3]) \\ \equiv\ & \exists q_1, q_2, q_3.\ 6 \hookrightarrow 1 \star 7 \hookrightarrow q_1 \star q_1 \hookrightarrow 2 \star (q_1+1) \hookrightarrow q_2 \star q_2 \hookrightarrow 3 \star (q_2+1) \hookrightarrow q_3 \star [q_3 = 0] \end{aligned}$$

The heap above satisfies the last line, with $q_1 = 30$, $q_2 = 14$, and $q_3 = 0$.

What $\star$ rules out. Because each unfolding claims new cells that are disjoint from all the others, and the list must end at 0, no list L makes $\text{list}(p, L)$ describe a cyclic structure. Similarly, $\text{list}(p_1, L_1) \star \text{list}(p_2, L_2)$ guarantees that the two lists share no nodes: a shared tail would have to belong to both halves of the heap. In Hoare logic we would need explicit "acyclic" and "disjoint" predicates to say these things. Here $\star$ gives us both for free.

Further Reading

Interested students are referred to two canonical treatments: Reynolds's lecture notes, or Charguéraud's habilitation thesis or textbook *Separation Logic Foundations* (Software Foundations, Volume 6), which this lecture is loosely based on.

In proofs, it will often be convenient to have structural rules that allow us to rearrange assertions in various ways. The Exists rule lets us reason about a command under an existential quantifier: if c takes H to J for every value of the logical variable i, then it takes $\exists i.\, H$ to $\exists i.\, J$.

$$\frac{\vdash \{H\}\ c\ \{J\}}{\vdash \{\exists i.\, H\}\ c\ \{\exists i.\, J\}}\ \textsc{Exists}$$

In the examples, we will use this rule constantly, because unfolding a recursive predicate will always introduces existentials.

4.10 Example: Disposing a List

The following program dispose frees every node of the list at p:

```
while (p ≠ 0) do (
  q := *(p + 1);
  free(p);
  free(p + 1);
  p := q
)
```

Its specification is $\{\text{list}(p, L)\}$ dispose $\{\text{emp}\}$, and a suitable loop invariant is $\exists L'.\, \text{list}(p, L')$: the heap always holds exactly the rest of the list. The precondition implies the invariant (take $L' = L$). For the loop body, the While rule requires us to show $\{[p \neq 0] \star \exists L'.\, \text{list}(p, L')\}$ body $\{\exists L'.\, \text{list}(p, L')\}$:

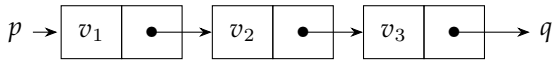
$\{(\exists L'.\, \text{list}(p, L')) \star [p \neq 0]\} \Rightarrow$ (unfold)
$\{\exists v, y, L''.\ p \hookrightarrow v \star (p+1) \hookrightarrow y \star \text{list}(y, L'')\}$
$q := *(p + 1);$ (Load, Frame)
$\{\exists v, y, L''.\ [q = y] \star p \hookrightarrow v \star (p+1) \hookrightarrow y \star \text{list}(y, L'')\}$
free(p); (Free, Frame)
$\{\exists y, L''.\ [q = y] \star (p+1) \hookrightarrow y \star \text{list}(y, L'')\}$
free$(p + 1)$; (Free, Frame)
$\{\exists y, L''.\ [q = y] \star \text{list}(y, L'')\} \Rightarrow$
$\{\exists L'.\, \text{list}(q, L')\}$
$p := q$ (assignment)
$\{\exists L'.\, \text{list}(p, L')\}$

The first implication unfolds the list: since $p \neq 0$, the list cannot be empty (as $\text{list}(p, [\,])$ would force $p = 0$), so $L' = v :: L''$ for some v and L''. Each command is then verified under the existential using Exists, and with Frame to carry along the cells it does not touch. For example, the load uses the Load axiom on the single cell $(p+1) \hookrightarrow y$ and frames $p \hookrightarrow v \star \text{list}(y, L'')$; its side condition holds because q occurs in neither $p + 1$ nor y. After **free**(p), no cell mentions v any more, so we drop $\exists v$ by Consequence. The last implication repackages what remains as the invariant with q in place of p, and the assignment axiom then gives the invariant itself.
When the loop exits, we have $(\exists L'.\, \text{list}(p, L')) \star [p = 0]$. Since $0 \hookrightarrow v$ is unsatisfiable, $\text{list}(0, L')$ forces $L' = [\,]$, and so the heap is empty, as required. Because triples are partial correctness statements, we did not need to prove that the loop terminates (although it does, since the length of L' decreases on every iteration).

4.11 List Segments

For loops that traverse a list without destroying it, we need to describe the part of the list that has already been visited. A *list segment* from p to q holding L is a list that ends at q instead of at 0:

$$\begin{aligned} \text{lseg}(p, q, [\,]) &\triangleq [p = q] \\ \text{lseg}(p, q, v :: L) &\triangleq \exists r.\ p \hookrightarrow v \star (p+1) \hookrightarrow r \star \text{lseg}(r, q, L) \end{aligned}$$



$$\mathrm{lseg}(p, q, [v_1, v_2, v_3])$$

The following facts are each proved by induction on L:

$$\mathrm{list}(p, L) \equiv \mathrm{lseg}(p, 0, L)$$

$$\mathrm{lseg}(p, x, L) \star x \hookrightarrow v \star (x+1) \hookrightarrow y \Rightarrow \mathrm{lseg}(p, y, L +\!\!+ [v]) \qquad \text{(snoc)}$$

$$\mathrm{lseg}(p, y, L) \star \mathrm{list}(y, L') \Rightarrow \mathrm{list}(p, L +\!\!+ L') \qquad \text{(concat)}$$

Snoc extends a segment by one node at its end, and concat attaches a list to the end of a segment.

4.12 Example: List Append

The following program append, appends the list at p_2 to the list at p_1, leaving the result at r.

```
x := p1;
n := *(x + 1);
while (n ≠ 0) do (
   x := n;
   n := *(x + 1)
);
*(x + 1) := p2;
r := p1
```

In the program, the variable x always points at a node of the first list, and n holds that node's next pointer; the loop walks x to the last node, and the final store overwrites its null pointer with p_2.
For example, suppose the heap initially holds the lists $[1, 2]$ at address 6 and $[3, 4]$ at address 99:

address	6	7	22	23	46	47	99	100
before	1	46	4	0	2	0	3	22
after	1	46	4	0	2	99	3	22

with $p_1 = 6$ and $p_2 = 99$. Afterwards, $x = 46$ (the last node of the first list), $n = 0$, $r = 6$, and the heap holds the list $[1, 2, 3, 4]$ at r. Only one cell changes—the last node is modified from 0 to p_2.
The specification for this program

$$\{\mathrm{list}(p_1, L_1) \star \mathrm{list}(p_2, L_2) \star [p_1 \neq 0]\}\ \mathrm{append}\ \{\mathrm{list}(r, L_1 +\!\!+ L_2)\}.$$

Two aspects of this specification deserve comment. First, the pure fact $p_1 \neq 0$ is essential: if the first list is empty, then $x = 0$, so the program accesses address 1, which need not be allocated, and it may produce a memory error. Second, $\mathrm{list}(p_2, L_2)$ does not appear in the postcondition because its cells have become part of $\mathrm{list}(r, L_1 +\!\!+ L_2)$. The precondition's $\star$ guarantees that the two lists are disjoint, so appending cannot create a cycle.
To keep the proof readable, we abbreviate a node as

$$p{:}\langle v \mid q \rangle \ \triangleq\ p \hookrightarrow v \star (p+1) \hookrightarrow q.$$

The proof has three parts: the code before the loop, which establishes the invariant; the loop body, which preserves it; and the code after the loop, which establishes the postcondition.

Before the loop. We frame $\mathrm{list}(p_2, L_2)$, which this code does not touch. Since $p_1 \neq 0$, the first list is not empty, so we can unfold its first node:

$$
\begin{array}{l}
\{\, \mathrm{list}(p_1, L_1) \star [p_1 \neq 0] \,\} \;\Rightarrow \\
\{\, \exists v, y, L_b.\ [L_1 = v :: L_b] \star p_1{:}\langle v \mid y \rangle \star \mathrm{list}(y, L_b) \,\} \\
x := p_1; \\
\{\, \exists v, y, L_b.\ [L_1 = v :: L_b] \star [x = p_1] \star x{:}\langle v \mid y \rangle \star \mathrm{list}(y, L_b) \,\} \\
n := *(x + 1); \\
\{\, \exists v, y, L_b.\ [L_1 = v :: L_b] \star [x = p_1] \star [n = y] \star x{:}\langle v \mid y \rangle \star \mathrm{list}(y, L_b) \,\} \;\Rightarrow \\
\{\, \exists v, L_b.\ [L_1 = [\,] +\!\!+ v :: L_b] \star \mathrm{lseg}(p_1, x, [\,]) \star x{:}\langle v \mid n \rangle \star \mathrm{list}(n, L_b) \,\}
\end{array}
$$

Note that the pure fact $[L_1 = v :: L_b]$ must be carried through every assertion: it is the only connection between the logical variables v and L_b and the list L_1 in the specification. In the last step we use $\mathrm{lseg}(p_1, x, [\,]) \equiv [p_1 = x]$ and replace y by n. Generalizing $[\,]$ to an arbitrary L_a, and adding back the frame, suggests the loop invariant

$$I \;\triangleq\; \exists L_a, v, L_b.\ [L_1 = L_a +\!\!+ v :: L_b] \star \mathrm{lseg}(p_1, x, L_a) \star x{:}\langle v \mid n \rangle \star \mathrm{list}(n, L_b) \star \mathrm{list}(p_2, L_2).$$

The visited part of the first list is the segment from p_1 to x; x is the current node, holding v and pointing to n; and the rest of the first list is at n. The assertion we derived is exactly I with $L_a = [\,]$, so the code before the loop establishes the invariant.

The loop body. We must show $\{I \star [n \neq 0]\}\ x := n;\ n := *(x+1)\ \{I\}$. We again frame $\mathrm{list}(p_2, L_2)$. Since $n \neq 0$, the rest of the list is not empty, so we unfold $L_b = w :: L_c$:

$$
\begin{array}{l}
\{\, I \star [n \neq 0] \,\} \;\Rightarrow\; \text{(unfold)} \\
\{\, \exists L_a, v, w, m, L_c.\ [L_1 = L_a +\!\!+ v :: w :: L_c] \\
\qquad \star\, \mathrm{lseg}(p_1, x, L_a) \star x{:}\langle v \mid n \rangle \star n{:}\langle w \mid m \rangle \star \mathrm{list}(m, L_c) \,\} \;\Rightarrow\; \text{(snoc)} \\
\{\, \exists L_a, v, w, m, L_c.\ [L_1 = L_a +\!\!+ v :: w :: L_c] \\
\qquad \star\, \mathrm{lseg}(p_1, n, L_a +\!\!+ [v]) \star n{:}\langle w \mid m \rangle \star \mathrm{list}(m, L_c) \,\} \\
x := n; \qquad \text{(assignment)} \\
\{\, \exists L_a, v, w, m, L_c.\ [L_1 = L_a +\!\!+ v :: w :: L_c] \\
\qquad \star\, \mathrm{lseg}(p_1, x, L_a +\!\!+ [v]) \star x{:}\langle w \mid m \rangle \star \mathrm{list}(m, L_c) \,\} \\
n := *(x + 1) \qquad (\textsc{Load}, \textsc{Frame}) \\
\{\, \exists L_a, v, w, m, L_c.\ [L_1 = L_a +\!\!+ v :: w :: L_c] \star [n = m] \\
\qquad \star\, \mathrm{lseg}(p_1, x, L_a +\!\!+ [v]) \star x{:}\langle w \mid m \rangle \star \mathrm{list}(m, L_c) \,\} \;\Rightarrow \\
\{\, I \,\}
\end{array}
$$

The snoc step folds the current node into the visited segment. The final implication re-establishes I with $L_a +\!\!+ [v]$ for L_a, w for v, and L_c for L_b. This fact holds because $L_a +\!\!+ v :: w :: L_c = (L_a +\!\!+ [v]) +\!\!+ w :: L_c$, and $x{:}\langle w \mid m \rangle \star \mathrm{list}(m, L_c)$ becomes $x{:}\langle w \mid n \rangle \star \mathrm{list}(n, L_c)$ because $n = m$.

After the loop. When the loop exits, we have $I \star [n = 0]$. Since $n = 0$, the rest of the list must be empty, so $L_b = [\,]$ and hence $L_1 = L_a +\!\!+ [v]$:

$$
\begin{array}{l}
\{\, I \star [n = 0] \,\} \;\Rightarrow \\
\{\, \exists L_a, v.\ [L_1 = L_a +\!\!+ [v]] \star \mathrm{lseg}(p_1, x, L_a) \star x{:}\langle v \mid 0 \rangle \star \mathrm{list}(p_2, L_2) \,\} \\
*(x + 1) := p_2; \qquad (\textsc{Store}, \textsc{Frame}) \\
\{\, \exists L_a, v.\ [L_1 = L_a +\!\!+ [v]] \star \mathrm{lseg}(p_1, x, L_a) \star x{:}\langle v \mid p_2 \rangle \star \mathrm{list}(p_2, L_2) \,\} \;\Rightarrow\; \text{(snoc)} \\
\{\, \mathrm{lseg}(p_1, p_2, L_1) \star \mathrm{list}(p_2, L_2) \,\} \\
r := p_1 \qquad \text{(assignment)} \\
\{\, \mathrm{lseg}(r, p_2, L_1) \star \mathrm{list}(p_2, L_2) \,\} \;\Rightarrow\; \text{(concat)} \\
\{\, \mathrm{list}(r, L_1 +\!\!+ L_2) \,\}
\end{array}
$$

This establishes the postcondition and completes the proof.

4.13 The Magic Wand

In standard Hoare logic, the assignment axiom $\{P[e/x]\}\ x := e\ \{P\}$ works backwards: it computes the weakest precondition of an assignment by substitution. What is the weakest precondition of a heap store such as $*x := 5$ with respect to a postcondition Q? Substitution doesn't work, because Q may refer to the cell at x under a different name—e.g., an assertion such as $y \hookrightarrow 3$ may refers to the same cell when x and y are aliases. What we would like to say is: "I hold the cell at x, together with a promise: add $x \hookrightarrow 5$, and you get Q." Our existing assertion language has no way to express such a promise, so we add a new connective, called *magic wand*.

Definition. The *magic wand*, or *separating implication*, is defined by

$$(\sigma, h) \models_I H \mathrel{-\!\!\star} J \quad \text{iff for all } h' \perp h \text{ with } (\sigma, h') \models_I H, \text{ we have } (\sigma, h \uplus h') \models_I J.$$

Intuitively, $H \mathrel{-\!\!\star} J$ describes a heap that becomes a J-heap when you add an H-heap; it is sometimes described as "J with an H-shaped hole." The wand is to $\star$ as implication is to conjunction. In particular, it satisfies modus ponens and a currying law:

$$H \star (H \mathrel{-\!\!\star} J) \Rightarrow J \qquad\qquad (K \star H \Rightarrow J) \text{ iff } (K \Rightarrow H \mathrel{-\!\!\star} J)$$

For example, $\mathrm{emp} \mathrel{-\!\!\star} H \equiv H$, and any heap satisfying $y \hookrightarrow 4$ also satisfies

$$x \hookrightarrow 3 \mathrel{-\!\!\star} (x \hookrightarrow 3 \star y \hookrightarrow 4).$$

(If x and y are aliases, no disjoint heap satisfies $x \hookrightarrow 3$, so the wand holds vacuously.)

Weakest preconditions for heap commands. With the wand, we can state the weakest preconditions of the heap commands, where "weakest" is with respect to our fault-avoiding triples:

Command	**Weakest precondition for Q**
$*e_1 := e_2$	$(e_1 \hookrightarrow -) \star ((e_1 \hookrightarrow e_2) \mathrel{-\!\!\star} Q)$
$\mathbf{free}(e)$	$(e \hookrightarrow -) \star Q$
$x := \mathbf{new}(e)$	$\forall i.\ (i \hookrightarrow e) \mathrel{-\!\!\star} Q[i/x] \quad (i \text{ fresh})$

For a store, we must own the cell at e_1 (so that the store does not fault), and the rest of the heap must turn into a Q-heap once a cell $e_1 \hookrightarrow e_2$ is added. For allocation, the rest of the heap must turn into a Q-heap once any new cell $i \hookrightarrow e$ is added, with x holding its address i, because New may choose any unused address. For example, with $Q = x \hookrightarrow 5 \star y \hookrightarrow 7$,

$$\mathrm{wp}(*x := 5,\ Q) = x \hookrightarrow - \star (x \hookrightarrow 5 \mathrel{-\!\!\star} (x \hookrightarrow 5 \star y \hookrightarrow 7)) \iff x \hookrightarrow - \star y \hookrightarrow 7,$$

by currying, since $y \hookrightarrow 7 \star x \hookrightarrow 5 \Rightarrow x \hookrightarrow 5 \star y \hookrightarrow 7$. This is exactly what we would expect.

Backward rules from forward ones. We do not need new proof rules to use these weakest preconditions. For example, the backward rule for stores follows from the forward Store axiom. Let $K = (e_1 \hookrightarrow e_2) \mathrel{-\!\!\star} Q$. Since $\mathrm{mod}(*e_1 := e_2) = \emptyset$, the Store axiom and Frame give

$$\{e_1 \hookrightarrow - \star K\}\ *e_1 := e_2\ \{e_1 \hookrightarrow e_2 \star K\}.$$

By modus ponens, $e_1 \hookrightarrow e_2 \star ((e_1 \hookrightarrow e_2) \mathrel{-\!\!\star} Q) \Rightarrow Q$, so by Consequence,

$$\{(e_1 \hookrightarrow -) \star ((e_1 \hookrightarrow e_2) \mathrel{-\!\!\star} Q)\}\ *e_1 := e_2\ \{Q\}.$$

In other words, the forward small axioms together with Frame already suffice; the wand simply lets us state weakest preconditions.

Where the magic wand shows up. The wand plays a central role in modern program logics. In Iris, for example, the rules for weakest preconditions are stated using $\mathrel{-\!\!\star}$, much like the backward rules above. Viper supports magic wands in specifications. The wand is also useful for describing partial data structures: for instance, $\mathrm{list}(x, L_2) \mathrel{-\!\!\star} \mathrm{list}(p, L_1 +\!\!+ L_2)$ describes a list with a hole, which becomes a complete list once the list at x is supplied. On the other hand, the wand is notoriously hard to automate, which is one reason why most tools reason in the forward direction, often using symbolic execution.

Further Reading

Reynolds's lecture notes cover list segments, the structural rules (including the form of the EXISTS rule used here), and backward reasoning with the magic wand. Charguéraud's textbook *Separation Logic Foundations* (Software Foundations, Volume 6) takes as primitive a variant of the EXISTS rule with the existential only in the precondition (`triple_hexists`), and derives the form used here from it (`triple_hexists2`).